AccDSL

Company Histories, Accounting Interpretations, and Scenario Simulation

Product and implementation design

Version 1.0 2 October 2026

This document specifies a system that translates company disclosures into traceable evidence, represents several compatible economic and accounting histories, and simulates their future journals and financial statements under explicit economic scenarios.

The core requirements, data contracts, semantics and acceptance cases govern implementation. Proposed libraries can be replaced while preserving those contracts. Sector rule packages require their own accounting review before they can be described as compliant with a reporting standard.

Research materials, source qualifications and review records accompany this document in `docs/design/research` and `docs/design/review`.

Contents

1 The product decision **1**

1.1 Why journals are necessary but insufficient 1

1.2 Users, questions and deliverables 2

1.3 What constitutes a useful first implementation 3

2 Representing a company's historical record **4**

2.1 The record is a collection of assertions 4

2.2 Different names, different measurements 5

2.3 Time, revisions and information availability 5

2.4 The company boundary changes 6

2.5 Further problems that the representation must handle 6

3 What the literature contributes **8**

3.1 Accounting structure and company dynamics 8

3.2 Incomplete information and provenance 9

3.3 Inverse problems, identification and probability 10

3.4 Languages for rules and contracts 10

3.5 Extraction and financial reasoning 11

3.6 Solvers and probabilistic execution 12

3.7 The remaining design problem 12

4 Architecture and reuse **13**

4.1 One representation, several authoring methods 13

4.2 Why a DSL, and when another approach is better 14

4.3 Open-source components 15

4.4 Responsibilities that AccDSL must implement 16

5 Historical compatibility and future simulation **17**

5.1 Define what is observed before inverting it 17

5.2 Why published statements cannot identify all transactions 18

5.3 The accounting state and its invariants 18

5.4 Simulate events from economic state . . . 19
5.5 Preserve sets and distributions separately . . . 19
5.6 Resolution and computational limits . . . 20

6 The accounting language 22
6.1 Scope of the first language . . . 22
6.2 Types that protect meaning . . . 22
6.3 A small model with a continuous interpretation family . . . 23
6.4 Posting rules and explicit exceptions . . . 24
6.5 Alternative readings, assumptions and updates . . . 25
6.6 Modules and stable intermediate representation . . . 25
6.7 Errors are part of the interface . . . 26

7 Reconstruction and execution procedures 27
7.1 Ingest, reconcile and construct histories . . . 27
7.2 Condition a run without rewriting history . . . 28
7.3 Schedule and commit events . . . 28
7.4 Funding, tax and other simultaneous relations . . . 29
7.5 Worked example: identical history, different future journals . . . 29
7.6 Worked example: a bond has several relevant amounts . . . 30
7.7 Reporting and explanations . . . 30

8 Standards, industries and financing instruments 32
8.1 A common core with distinct accounting books . . . 32
8.2 Non-financial companies . . . 33
8.3 Debt and hybrid financing . . . 33
8.4 Equity and ownership . . . 34
8.5 Banks: accounting capital is not regulatory capital . . . 34
8.6 Insurance: cohorts and future service . . . 35
8.7 Capability contracts for sector packages . . . 36

9 Implementation contracts 37
9.1 Reference implementation and component boundaries . . . 37
9.2 Persistent records . . . 37
9.3 Public API and result types . . . 39
9.4 Plugin interfaces . . . 39
9.5 Human review and model-assisted extraction . . . 39
9.6 Failure recovery and reproducibility . . . 40

10 Acceptance and implementation sequence **41**

10.1 Three different questions require different evidence 41

10.2 Core conformance cases 41

10.3 Extraction and reconstruction evaluation 43

10.4 Evaluate the DSL against the strongest practical baseline 43

10.5 Forecasting evaluation comes later 44

10.6 Implementation milestones and dependency order 44

10.7 Decisions and replaceable assumptions 45

A Core grammar, records and execution details **46**

A.1 Core 0 grammar 46

A.2 Required core record definitions 47

A.3 Typing and partial-information rules 49

A.4 Operational rule selection 49

A.5 Example run request and response 50

A.6 Requirement traceability 50

B Research basis and outstanding review **51**

B.1 Source collection and reading scope 51

B.2 Version and access qualifications 52

B.3 Companion records and status 53

Chapter 1

The product decision

A company's accounts describe the financial consequences of its activities, but they leave much of the activity itself unobserved. Revenue combines many sales, a borrowing balance combines contracts with different terms, and a provision combines estimates about uncertain future payments. Aggregation is necessary for reporting. It also makes the inverse task—recovering economic history from published reports—fundamentally different from entering known transactions into a ledger.

AccDSL will preserve this distinction. Its first product is an evidence-backed representation of a company's historical record. Its second is a set of compatible histories and current economic states. Its third is a collection of conditional simulations, each explaining how a particular history, accounting treatment and business response produces future journals and reports. These products share identifiers and provenance, so an analyst can move from a forecast balance to the assumption or disclosure responsible for it.

The recommended design is a small, typed accounting language over a shared intermediate representation, combined with an evidence database, contract calculators and replaceable inference engines. A language earns its place by making recurring accounting decisions explicit and checkable: which entity and period an amount belongs to, whether it is known or assumed, which rule applies, and how an event changes a balanced ledger. The language does not supply missing facts or determine a company's future behaviour.

1.1 Why journals are necessary but insufficient

Consider a company with a borrowing of 100 currency units and annual interest expense of 5. Assume that the entire principal was outstanding throughout the year, interest was paid at year-end, and there were no fees, defaults, hedges or foreign-exchange effects. Both a fixed-rate loan at 5% and a floating-rate loan whose rate happened to be 5% generate the same entries:

Account	Debit	Credit
Cash, on initial borrowing	100	
Borrowing liability		100
Interest expense, at year-end	5	
Cash		5

Even the full historical journal would not distinguish these two contracts. Now impose a scenario in which the floating rate resets immediately to 7% for the whole next year. The fixed

loan produces interest of 5; the floating loan produces interest of 7. If the disclosure is also compatible with any fixed-rate fraction $\alpha \in [0, 1]$, future interest is

$$I(\alpha) = 100[0.05\alpha + 0.07(1 - \alpha)] = 7 - 2\alpha. \tag{1.1}$$

The historical accounts are compatible with the whole interval $[5, 7]$ of future interest. This is a range of feasible amounts. It becomes a probability distribution only when a distribution for α is supplied. Neither the number of generated interpretations nor a language model's confidence provides that distribution.

Journals remain essential because simulated economic events must have coherent accounting consequences. However, the simulated state must also contain contract terms, maturities, customer or borrower cohorts, collateral, unrecognised commitments, and management decisions. These determine what the journals will be under a changed economy. Reconstructing every historical transaction is unnecessary when a smaller state preserves the responses needed for the intended scenarios. Establishing that smaller state is an explicit modelling task, examined in Chapter 5.

The analogy with competing legal interpretations is useful at the level of representation. Several readings may survive the evidence and require separate reasoning. In accounting, some alternatives concern the meaning of a disclosure; others concern undisclosed facts or future behaviour. The system must identify which kind of uncertainty is involved. A known contractual term must not become an arbitrary interpretive choice merely because another choice is convenient for a simulation.

1.2 Users, questions and deliverables

An accounting analyst needs to inspect a number in its original table, verify its entity, currency and period, and see how it reconciles with the notes. A financial modeller needs to distinguish a reported balance from assumptions about its composition. A language engineer needs deterministic semantics and tests. A researcher needs to compare explanations under the same scenario, including explanations that fit history equally well.

The first release must answer the following complete workflow. An analyst loads a set of reports and specifies an information cutoff. AccDSL preserves the original documents and extracts source-linked observations. It identifies reconciliation failures and constructs an explicitly bounded family of compatible histories. The analyst declares additional assumptions where simulation requires them. The system executes each requested scenario and returns statements, journal traces, explanation-specific distributions or bounds, and unresolved limitations. When a later filing arrives, the analyst can compare the revised interpretation with the earlier information set.

Requirement	Required behaviour
R1 Evidence	Every observed amount or narrative assertion retains its document version, location, extraction method and original representation.
R2 History	Queries distinguish economic time, reporting period, publication time and database ingestion time; restatements remain recoverable.
R3 Meaning	Entity, perimeter, dimensions, currency, measurement basis, standard version and accounting concept accompany values.

Requirement	Required behaviour
R4 Ambiguity	Discrete alternatives and continuous unknowns retain their joint constraints across accounts and periods. Missing values never mean zero.
R5 Accounting	All committed journals balance in the book's functional currency, and balances reconcile to explicitly classified movements.
R6 Economics	Contracts, exposures and behavioural assumptions drive events. Accounting identities alone never determine scenario responses.
R7 Results	Conditional results survive by interpretation and scenario. Weighted results require declared, normalised probability measures.
R8 Explanation	A report cell can be traced to journals, events, rules, assumptions and evidence, including consolidation adjustments.
R9 Extensibility	Standard, industry and company packages extend shared types through versioned interfaces, with explicit unsupported cases.
R10 Reproduction	A saved run records source versions, model versions, parameters, calendars, seeds and numerical conventions.

These are acceptance requirements. Python, SQL storage, a particular parser, and a particular constraint solver are implementation choices. Initial support will cover a non-financial company with cash, receivables, payables, inventory, fixed assets, ordinary debt and ordinary equity. Banks and insurers must be representable in the core data model from the outset, but their executable measurement and behavioural packages follow dedicated milestones. Until a package supports an instrument, the result is explicitly incomplete.

1.3 What constitutes a useful first implementation

The first demonstrator should reconstruct three reporting dates and the intervening annual and interim flows for one company, including at least one restatement or revised comparative. It must preserve a borrowing-composition ambiguity and execute the fixed/floating example through both journals and all affected statement lines. A synthetic company supplies known hidden history; a public company supplies realistic disclosure and extraction problems. Their roles differ: the synthetic case tests preservation and rejection of known alternatives, while the public case tests traceability and practical coverage.

Success is a reproducible chain of evidence and computation, with identified gaps. A convincing-looking complete ledger reconstructed from incomplete disclosures would fail this product contract if its assumptions were hidden. Forecast accuracy requires a later evaluation with historical information cutoffs and suitable baselines; it is not established by the first demonstrator.

Chapter 2

Representing a company's historical record

2.1 The record is a collection of assertions

The original record includes financial statements, notes, accounting policies, auditor opinions, management discussion, segment tables, security terms, regulatory returns and subsequent announcements. A complete ingestion process accounts for every page and every extracted block. A complete financial model is a stronger claim: it would require economically relevant information that public reports may never disclose. AccDSL therefore reports document coverage, semantic coverage and simulation coverage separately.

Each source span receives a disposition: structured observation, narrative assertion, relationship, policy evidence, contextual material, duplicate, or unresolved extraction. "Unresolved" remains visible. A commitment buried in a note is not discarded because it has no current balance-sheet line. An auditor's qualification is retained as evidence about the record; it does not become an automatic numerical haircut. Management's forecast is a dated assertion with an author and modality, not an observed future outcome.

An observation is an immutable record with the fields below. Values inferred from observations are separate records, linked by a derivation. This avoids overwriting a company's statement with an analyst's preferred reconstruction.

Field group	Minimum contents
Identity and source	Observation ID; document hash; filing identifier; PDF page and bounding box or HTML/XBRL locator; literal text; extraction version.
Reporting context	Legal entity; reported group perimeter; statement or note; instant or interval; standard and taxonomy versions; dimensions such as segment, product, geography, seniority or measurement category.
Amount	Original string; parsed number; sign interpretation; unit; scale; currency; precision or rounding interval; missing-value category.
Meaning	Source label; taxonomy QName when present; one or more proposed concept mappings; whether the assertion is reported, estimated or conditional.

Field group	Minimum contents
Timing	Economic validity when known; reporting period; publication and availability timestamps; ingestion timestamp; correction or supersession links.
Reliability and review	Extraction confidence, reviewer decision, source status and known contradictions, all distinct from a probability over histories.

A narrative assertion adds actor, predicate, object, scope, quantifier and modality. For example, "substantially all debt is fixed rate" has a source span and a qualitative quantifier. It does not assert $\alpha = 1$. A reviewed interpretation can introduce a numerical restriction such as $\alpha \geq a$, but the threshold a remains an analyst assumption unless the source defines it. Boilerplate policy language also does not prove that a company holds a particular instrument.

2.2 Different names, different measurements

An accounting concept is more than a row label. "Debt" can refer to legal principal, amortised carrying amount, undiscounted contractual cash flows or a fair-value disclosure. These amounts can all be correct at the same date. Likewise, "revenue" may be reported gross or net, a segment measure may differ from the consolidated measure, and an alternative performance measure may exclude selected expenses. A mapping must state its relationship: equivalent meaning, component of, broader than, presentation reclassification, or measurement transformation.

The common vocabulary will preserve source concepts as first-class identifiers. Company extensions may inherit a broad economic category without being declared identical to a standardised concept. For instance, an extension can be a component of borrowings while retaining an unresolved split between lease liabilities and other financing. Many-to-many mappings are allowed; each mapping has applicable dates, dimensions and evidence. An aggregation mapping does not silently become a measurement conversion.

Standards differ in recognition, measurement, presentation and disclosure. Consequently, conversion between IFRS, US GAAP and Chinese Accounting Standards for Business Enterprises (ASBE) is not generally a dictionary lookup. The original basis remains preserved, and any comparison basis is a separate derived book. The same principle applies to tax and prudential capital. A tax loss carryforward, accounting deferred tax asset and regulatory capital deduction can concern the same economic loss while belonging to different measurement systems.

2.3 Time, revisions and information availability

At least four times matter. A machine may enter service in November; its cost appears in the accounts for the year ending December; the annual report is published in March; and AccDSL ingests the report in April. A later report may revise the December comparative. These times answer different questions and must remain distinct.

The storage model uses validity intervals for economic assertions and transaction intervals for database versions, following the distinction in Snodgrass and Ahn (1986). Publication and availability timestamps are additional domain fields. Database transaction time records what AccDSL contained; it cannot substitute for what investors could have known. An "as known on" query applies an explicit availability rule and does not include a later restatement simply because its economic period is earlier.

AccDSL supports three named views: the original filing view; the latest restated view, with a stated information cutoff; and a harmonised research view, with explicit transformations. No view changes the stored source facts. Corrections have links to the replaced assertion and a reason. Where two sources conflict without a clear supersession relationship, both survive with a conflict record.

Instant values and interval flows have different types. A balance at 30 June is not a six-month flow. Calendar intervals use half-open boundaries $[t_0, t_1)$ internally; a filing period ending 30 June maps to the next boundary at 1 July in its reporting calendar. Quarter-only earnings may be derived from year-to-date values only if the periods, basis, perimeter, units and revision vintage are compatible. The derivation retains both source observations and their correlated rounding uncertainty. Fifty-two/fifty-three-week calendars, stub periods and reporting-date changes require explicit calendar objects.

Accounting-policy changes, estimate changes and error corrections also have different temporal effects. The applicable rule package decides whether a change is prospective, requires restatement, or adjusts opening balances. The engine never infers that effect from a generic "revision" flag.

2.4 The company boundary changes

Legal entity, reporting group and economic exposure are separate dimensions. Acquisitions, disposals, changes in control, joint arrangements, special-purpose vehicles and non-controlling interests can change the reporting perimeter. A group's increase in property need not be capital expenditure; it may reflect an acquisition, a currency translation or a revaluation. The bridge between opening and closing property must identify these movements before a scenario model can extrapolate investment.

The company graph therefore stores dated ownership, control and counterparty relationships. Control is an accounting conclusion with supporting evidence, not a rule that blindly consolidates every majority-owned entity. Group reporting combines entity books with a separate adjustment and elimination book. Intragroup receivables and payables require matching counterparties, currencies and dates. Unmatched eliminations remain unresolved; a balancing "plug" is not a recovered transaction.

Acquired opening balances, goodwill, purchase-price allocation, divested operations and non-controlling interests each have their own event types. Analyses can preserve reported scope or define a constant perimeter, but the latter is a derived modelling assumption, not the original record.

2.5 Further problems that the representation must handle

Several difficulties extend beyond differences in standards, industries and company labels. Recognition and settlement occur on different dates: a sale, invoice, cash receipt and impairment can belong to four periods. Net balances hide gross flows: repeated borrowing and repayment can leave the same closing debt. Netting can conceal offsetting exposures and liquidity needs. Different contractual maturities and repricing dates can share the same current carrying amount. Guarantees, undrawn credit facilities, litigation and covenants can be economically important before an asset or liability is recognised.

Foreign currency introduces transaction currency, functional currency and presentation currency, together with different translation conventions for stocks and flows. Currency translation

effects must not be mistaken for cash receipts. Inventory and depreciation depend on vintages or lots; an aggregate stock and average age may be insufficient under a shock. Tax bases can evolve differently from carrying amounts. Deferred tax recognition requires assumptions about recoverability, rather than multiplication by a tax rate alone.

Public information is selectively disclosed and rounded. Duplicate XBRL facts may agree, overlap within rounding bounds or contradict one another. A dash can mean zero, immaterial, unavailable or inapplicable. Tables can mix units, span pages and embed signs in parentheses. Text and table references can be wrong in the source itself. AccDSL retains these distinctions and gives an analyst a local repair path without pretending that extraction confidence settles economic interpretation.

Finally, strategic behaviour matters. A firm may refinance, cut investment, raise equity, change prices, breach a covenant or fail. A bank may preserve a capital ratio by shrinking assets; an insurer may change reinsurance or asset allocation. Such responses cannot be deduced from the balance-sheet identity. They require explicitly calibrated or assumed policies, and alternative policies must remain separate when the available record does not distinguish them.

Chapter 3

What the literature contributes

The research problem crosses accounting, databases, programming languages, econometrics and document understanding. Each field resolves part of it. The review below distinguishes results that can be adopted directly, engineering ideas that need adaptation, and evidence that does not establish the intended product capability. Source versions and technical anchors are recorded in the accompanying research ledgers. The search includes work outside the original collection, with public-index searches and backward and forward citation searches through 2 October 2026. It is a scoped design survey, rather than a claim to enumerate every paper in these fields.

3.1 Accounting structure and company dynamics

Nehmer and Robinson (1997), especially Sections 2–5, supplies a useful algebraic starting point: balances occupy a structured state space, transactions change that state, and aggregation relates detailed accounts to reports. This supports a distinction between conservation constraints and permitted transactions. An entry can conserve the accounting total and still be an inadmissible transaction. AccDSL adopts that distinction in its posting validator. Its particular event vocabulary, evidence model and inverse-history semantics are new design decisions, not results established by the algebraic paper.

Doshi (2024) develops an accounting DSL with time, transaction logs and correction mechanisms. The relevant material is in Chapters 4 and 7–9 of the 2024 thesis; the original collection's filename misleadingly suggests 2026. It demonstrates why corrections and dates belong in the language rather than in informal comments. The thesis does not establish recovery of an entire company history from public reports or inference over economically distinct histories. Babcinețchi et al. (2022), pp. 342–345, offers a much smaller accounting language and grammar proposal. It is useful as a syntax precedent, but it provides insufficient semantics or evaluation to govern AccDSL by itself.

Shahnazarian (2004) connects accounting dynamics with behavioural and econometric equations. Sections 2.3–2.8 describe the accounting relationships; the optimisation and econometric discussion in Chapter 3 and the data and evaluation material in Chapters 4 and 8 show how the model is used with standardised Swedish corporate data. This is an important precedent for simulating companies under changed conditions. Its standardised tax-data starting point differs from heterogeneous public-report extraction, and its working-capital funds formulation should not be imported as a general modern cash-flow statement implementation. The transferable idea is the explicit separation of accounting evolution and behavioural assumptions.

Pierson (2020), Sections 3–4 and the reporting structures, makes that separation operational in system dynamics. Receivables, payables and wages accumulate differences between accrual and payment flows; reporting-period accumulators turn instantaneous flows into statements. AccDSL can use these structures as examples of coherent movement schedules. The model's payment delays, inventory approximations and simplified equity treatment remain modelling choices. They are not universal defaults for companies or evidence that an aggregate state is sufficient for all shocks.

Bank models demonstrate why journal generation needs economic state. Halaj (2016), Section 3, combines funding shocks, fire sales, loans, securities and risk constraints. The homogeneous portfolios and specified funding process make the model tractable, but limit instrument-level interpretation. Behn et al. (2019), Section 2, makes managers' choices over asset growth, debt, risk and payouts depend on capital and liquidity constraints. These models motivate interfaces for management policy and regulatory calculations. Their parameterisations must be calibrated and evaluated for the target application; passing their accounting identities would not validate an AccDSL forecast.

Two important neighbouring traditions remain explicit coverage qualifications. McCarthy's 1982 REA model is a foundational candidate for the relation between resources, events and agents; Caiani and co-authors' stock-flow-consistent agent-based benchmark is a candidate for simulation across interacting sectors. Verified primary PDFs could not be obtained in this retrieval campaign after institutional and publisher attempts. Their records are retained in the omission register, and no technical result from either is used to justify the design. AccDSL's event and counterparty records are specified and justified directly below. A later multi-company equilibrium engine would require expansion of the stock-flow-consistent literature before implementation.

3.2 Incomplete information and provenance

Financial disclosures are incomplete databases in a substantive sense. Multiple underlying records can yield the same published observations. Libkin (2014), Sections 2–5, explains the semantics of incomplete databases, labelled nulls, conditional tables and certain answers. Repeated occurrences of the same unknown must share a value. Conditional tables can represent dependence between alternatives that independent missing cells cannot express. This is directly relevant to a debt note and a cash-flow table that refer to the same unknown repayment.

AccDSL adopts shared symbolic unknowns and conditions. It extends the database idea with accounting constraints, continuous amounts, temporal transitions and economic event models. A "certain" answer means a statement that holds for every history in the declared feasible family. The family itself may omit unknown mechanisms; certainty is therefore always relative to a model scope. Imieliński and Lipski's original 1984 paper is a foundational backward lead whose publisher copy was inaccessible in this campaign. The inspected Libkin paper supplies the technical discussion actually used here.

Green et al. (2007), Definitions 3.1–3.2 and Section 4, treats provenance as annotations that compose through relational operations. Combining sources and alternative derivations can therefore have formal semantics, rather than being a list of unrelated citations. This is useful for report aggregation and explanation queries. Positive relational operations covered by provenance semirings do not by themselves specify every operation AccDSL needs: negation, nonlinear valuation, optimisation and probabilistic conditioning require their own derivation records. Accordingly, the implementation starts with a typed dependency graph and may use semiring annotations for supported relational subqueries. A dependency edge is not a posterior

probability.

Snodgrass and Ahn (1986) distinguishes historical validity from the time facts are stored. The archived paper is *Temporal Databases*, by Snodgrass and Ahn, not the later Jensen–Snodgrass paper initially returned by a search. Its snapshot, rollback, historical and temporal examples support the need to recover both old business facts and old database beliefs. AccDSL adds public availability time because a database insertion date cannot establish whether a historical investor could have seen a filing.

3.3 Inverse problems, identification and probability

Stuart (2010), Section 2.2, starts from an observation equation $y = G(u)$ and distinguishes inversion, regularisation and Bayesian inference. The discussion makes explicit that a regularising norm or prior is an additional assumption. AccDSL has a similar observation problem, although its unknowns include discrete accounting interpretations and event structures as well as continuous parameters. The infinite-dimensional well-posedness theorems in that paper are not automatically applicable to this mixed, potentially discontinuous model.

The relevant issue is often identification rather than computation alone. Giacomini and Kitagawa (2021), Sections 2.1–2.6 and Theorem 1 of the archived 2020 working version, shows how a class of priors can preserve uncertainty within an identified set. A likelihood that is flat over the unresolved part does not learn the conditional prior over that part. Posterior lower and upper probabilities summarise a class of posterior beliefs. The paper's regularity conditions and application to structural vector autoregressions do not prove that an arbitrary accounting program admits the same algorithms or guarantees. The design borrows the distinction between learnable quantities and assumption-dependent distributions; the simple accounting bounds in Chapter 5 are derived directly.

Hoeting et al. (1999), equations (1)–(3), gives Bayesian model averaging as a weighted sum of model-conditional distributions, with weights determined by model priors and marginal likelihoods. Sections 3 and 5 discuss computation and prior specification. This supports an optional weighted mode, provided AccDSL specifies the likelihood, priors and candidate model space. It does not justify giving equal mass to every syntactically different journal reconstruction. Two encodings of the same economic history must not gain extra weight by duplication. Model averaging also remains conditional on the considered model family, rather than repairing missing mechanisms.

Scenario semantics need similar care. Pearl (2009), Section 3.2.1 and equations (5)–(6), distinguishes an intervention that replaces a structural equation from ordinary conditioning. AccDSL will distinguish a supplied exogenous path, an observational conditional forecast and an intervention. A rise in rates imposed by a stress test is not automatically distributed as the collection of historical periods in which rates happened to be high. Identifying a causal effect requires assumptions beyond reproducing accounts.

3.4 Languages for rules and contracts

van Deursen et al. (2000) surveys DSL benefits, risks and implementation options. Mernik et al. (2005), Sections 1.1 and 2.2–2.5 of the archived CWI report, treats application libraries as serious competitors and distinguishes deciding, analysing, designing and implementing a DSL. Its implementation decision tree explicitly includes embedding in a host language. The literature therefore supports a conditional design decision: a DSL is attractive when domain

constraints and repeated notation justify it, but an ordinary typed library may be cheaper and equally expressive. AccDSL will implement one shared representation accessible from both a Python API and a textual language, so syntax does not become a prerequisite for testing the accounting model.

Merigoux et al. (2021), Sections 3–4 and Figures 8–9, provides Catala's scopes, exceptions, default calculus and translation into a conventional functional target. Its explicit empty and conflict outcomes are valuable precedents for accounting rules with exceptions. They are different from probabilistic uncertainty. AccDSL will preserve this distinction: conflicting applicable rules produce a diagnostic, while an evidence-supported alternative reading produces a conditional model branch. The paper's compiler correctness results do not establish correctness of our accounting rules or of an automatic translation from disclosures.

The inspected LegalMath code offers reusable ideas in its shared typed interpretation, partial observations, source references and validity/knowledge times. Its ordinary frontend execution selects a reading. AccDSL needs an additional execution mode that retains a family of readings, continuous unknowns and economic state. A bounded candidate generator is useful for search, but its candidate limit does not prove completeness.

Peyton Jones et al. (2000), Sections 3–5, builds complex financial contracts from combinators and separates their compositional meaning from a chosen valuation model. The ACTUS technical specification similarly separates contract states, scheduled events, state transitions and payoffs, with risk factors supplied through an interface (ACTUS Financial Research Foundation, 2026). These are strong reasons to represent contractual obligations independently of accounting classifications. Neither contract cash flows nor a derivative price determine the recognition, measurement and presentation rules of a complete IFRS or US GAAP book.

3.5 Extraction and financial reasoning

The document literature supports a staged ingestion process. PubTables-1M (Smock et al., 2022), in its canonicalisation procedure and evaluation section, distinguishes table detection, structure recognition and functional analysis. Its GriTS evaluation compares reconstructed table structure. TableBank (Li et al., 2020), Sections 3–4, uses document structure to obtain weak supervision for table detection and recognition. GTE and FinTabNet (Zheng et al., 2021), Sections 3–5, provide a financial-document setting and company-level dataset splits. These sources help select extraction components and evaluation units. Their table scores do not establish that a debt note has been interpreted correctly across years or that every off-balance-sheet exposure has been found.

The Docling report (Auer et al., 2024), Section 3, describes PDF parsing, page layout analysis and table structure recognition within a document-conversion pipeline. The locally archived version lists Christoph Auer first; this version difference is recorded in the catalogue. Docling is an ingestion candidate whose output must retain source locations and undergo accounting validation. Its code and model licences must be considered separately when packaging a deployment.

FinQA (Chen et al., 2021), Sections 3–6, provides questions with executable reasoning programs and supporting evidence. TAT-QA (Zhu et al., 2021), Sections 2–4, combines table cells and text spans with aggregation operations, and explicitly treats numerical signs in evaluation. DocFinQA (Reddy et al., 2024) extends financial questions to long documents and evaluates retrieval/context choices. FinBen (Xie et al., 2024) broadens the collection of financial tasks and model evaluations. These benchmarks can test retrieval and local numerical reasoning. Their question-answer format does not require a complete multi-period company representation, joint

alternative histories or scenario-dependent journal generation.

A directly relevant recent competitor is the auditable RAG architecture of Cosentino et al. (2026). Sections 3.1–3.2 describe persistent extraction, enrichment, retrieval and provenance records; Section 4.6 checks citation resolvability and failure recovery. Section 5 limits the interpretation of those results to the evaluated document-intelligence tasks. AccDSL should reuse the idea of persistent evidence and recoverable processing, while adding executable accounting and economic semantics. No performance comparison with that architecture has been run here.

3.6 Solvers and probabilistic execution

de Moura and Bjørner (2008), Sections 2–3, describes Z3's interfaces and theory-solving architecture. It supports choosing an SMT solver for logical choices and arithmetic constraints, but does not make arbitrary nonlinear simulation tractable. The first solver fragment is deliberately bounded: finite choices, exact linear arithmetic and selected calendar relations. Unsupported formulas remain in the representation with a solver diagnostic.

van de Meent et al. (2018) develops probabilistic programs through sampling, observations and inference; the inspected material includes the first-order language and weighted sampling discussion. Stan (Carpenter et al., 2017), its model language and sampler-output discussion, and Pyro (Bingham et al., 2019), Sections 2–3, show different ways to separate a probabilistic model from inference machinery. They are candidates for optional inference over an already-defined economic model. Exact accounting identities and discrete rule selection should not be turned into nearly satisfied likelihood penalties merely to fit an inference library. Nor should successful sampler execution be described as evidence that all interpretations were explored.

3.7 The remaining design problem

The combined literature provides algebra for journals, semantics for missing information, temporal storage, provenance, contract languages, rule languages, inference methods and extraction tools. The unresolved integration problem is to retain their distinctions while connecting them: a disclosure constrains economic histories; a history supplies contract and behavioural state; a scenario drives events; accounting rules produce journals; reporting rules render statements; and every step remains explainable.

The design is therefore an integration proposal with several local mathematical constructions. Its originality and empirical advantage have not been established by this survey. Its practical merit will depend on the acceptance cases, accounting review and comparative implementation work in Chapter 10.

Chapter 4

Architecture and reuse

4.1 One representation, several authoring methods

The architectural centre is a typed intermediate representation (IR), not a text parser. A document extractor, a human editor, a Python library and the AccDSL compiler all create the same records. This allows a modeller to begin with a reviewed structured file and later add convenient notation. It also allows source extraction to improve without changing accounting semantics.

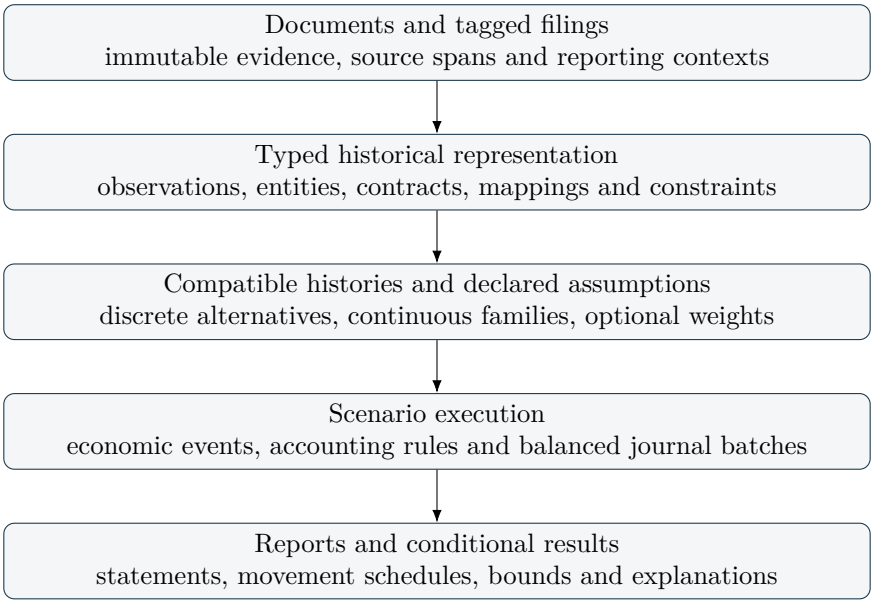


Figure 4.1: The simulation path. Every transformation records its inputs and assumptions; the source record remains available throughout.

Evidence storage and economic modelling must remain separate. If a company reports 100 of borrowing, a model can assume two loans of 60 and 40 without creating a new reported fact that the company disclosed those loans. The contract library consumes the assumed terms, the accounting module consumes the resulting events, and the explanation graph records the assumption at each affected result.

4.2 Why a DSL, and when another approach is better

The strongest baseline is a typed schema, SQL storage and ordinary Python code. That combination can implement the whole product. Its disadvantage is that repeated accounting requirements can become conventions scattered across functions: implicit currency conversion, missing period checks, hidden default values and duplicated rule selection. A bounded language makes these requirements part of typing and compilation. Its costs are language maintenance, diagnostics, migration and training.

Approach	Useful role	Limitation for this product
Typed schema and Python	Reference executor, flexible economic models, initial authoring API	Domain restrictions require disciplined APIs or a validator; arbitrary code complicates static analysis and provenance.
XBRL and OIM	Tagged evidence, contexts, dimensions, taxonomies and exchange	Reported facts and reporting relationships do not recover hidden economic histories or specify scenario behaviour.
Ledger languages	Recording known transactions, balance checks, export and differential testing	Alternatives, source interpretation and contract exposure need additional semantics.
Knowledge graph / SQL	Relationships, lineage, temporal queries and evidence navigation	A graph or schema alone does not define event execution or the probability meaning of alternatives.
Rules / Catala	Dated policy logic, defaults, exceptions, traceable rule evaluation	Ledger state, event scheduling and ensembles require an enclosing runtime.
Probabilistic program	Priors, likelihoods and inference over model parameters	Needs declared probability measures; cannot turn unspecified ambiguity into probability or supply accounting rules.
Spreadsheets	Familiar review and small transparent financial models	Versioned evidence, shared unknowns and branching histories are difficult to govern as the model grows.
AccDSL over shared IR	Common domain checks, reusable templates, explicit alternatives and deterministic posting semantics	Requires implementation and evaluation; broad notation without stable semantics would add cost.

The decision is to implement the IR, validators and Python reference API first, then the textual frontend. The same conformance cases must pass through both frontends. If the DSL fails to reduce ambiguity in representative authoring tasks, the IR and library remain useful. This is a reversible product decision, not a claim that a new language is inherently better than a library.

4.3 Open-source components

The following survey uses official repository descriptions, licence files and selected source interfaces archived on 2 October 2026. It is a source inspection, not an executed integration or performance evaluation. The machine-readable tool ledger records retrieval paths and available version metadata. Dependencies must be pinned when implementation begins; a branch name observed during this survey is not a release requirement.

Component	Proposed use and checked boundary	Licence / decision
Arelle	Parse and validate XBRL/iXBRL. Inspected `ModelFact`, context and unit accessors. Preserve taxonomy extensions and full contexts when importing.	Apache-2.0 core; first-choice adapter.
Docling	PDF layout, text and table conversion. Inspected converter interface; retain page coordinates and extraction settings.	MIT code; assess model licences separately. Candidate.
Table Transformer	Table extraction baseline associated with PubTables-1M; check finance-domain performance before selection.	MIT; repository marked archived in inspected metadata. Baseline, not a maintenance default.
Beancount	Posting and transaction data types; export selected concrete branches and compare balances.	GPL-2.0 project metadata; integration and redistribution choice needed. Optional adapter.
Ledger	Known-journal reference and export. Inspected transaction balancing code and manual date/assertion behaviour.	BSD-style licence; optional reference tool.
hledger	Alternative ledger import/export and accounting checks. Repository documentation inspected.	GPL-3.0 project metadata; optional process adapter.
Catala	Potential backend for deterministic policy modules; defaults and exceptions have useful semantics.	Apache-2.0; prototype only after core execution semantics stabilise.
ACTUS	Contract schedules, payoffs and transitions; external risk-factor interface. Verify event schedules against ACTUS tests.	Service Apache-2.0; technical specification CC-BY-SA-4.0. Candidate contract adapter.
QuantLib	Calendars, cash flows and valuation. Floating coupon source separates fixing dates and pricing. Accounting treatment remains external.	Modified BSD-style licence; candidate valuation adapter.
OpenFisca	Versioned tax-benefit formulas as a rule-system reference; inspected variable/formula metadata. Corporate-tax coverage must be supplied.	AGPL-3.0; optional, not the core runtime.

Component	Proposed use and checked boundary	Licence / decision
Z3	Finite alternatives, logical compatibility, linear constraints and conflict explanations.	MIT licence text; first solver candidate.
DuckDB	Local analytical queries over extracted facts and results.	MIT; candidate alongside SQLite transactional storage.
Stan	Continuous probabilistic models and diagnostics with an explicit target. Inspected model interface and paper.	BSD-3-Clause; optional inference backend.
Pyro / PyMC	Alternative probabilistic backends for model-specific inference. Pyro design paper and project descriptions inspected.	Apache-2.0 / Apache-2.0 licence texts; optional, no default selection.

The licence entries identify the inspected projects, not a legal conclusion about every combination of code, models and datasets. The default product architecture can communicate with optional tools through explicit adapters. This does not, by itself, settle redistribution obligations. That question is resolved when a concrete dependency and distribution method are selected.

XBRL's Open Information Model supplies standardised facts with concepts, entities, periods, units and dimensions (XBRL International, 2021). Those facts are an excellent ingestion source. AccDSL retains their original contexts and decimals rather than flattening them immediately into a company-by-year table. Calculation relationships can help validate reporting totals, but are not a general description of the transactions that generated those totals.

4.4 Responsibilities that AccDSL must implement

AccDSL owns the evidence and temporal model, typed accounting concepts, shared unknowns and alternatives, rule-package selection, journal validation, consolidation provenance and report explanations. It also owns the distinction between feasible sets and distributions. These responsibilities cannot be delegated implicitly to whichever parser or solver is convenient.

External components receive narrow requests and return typed responses. A contract adapter returns scheduled events, cash-flow quantities, state changes and source/model references; it does not directly mutate the ledger. A valuation adapter returns a measure, currency, date, method and numerical status; the accounting package decides whether and where to recognise the change. A solver returns feasibility or bounds with scope and completeness status. A language model proposes source-linked mappings or interpretations; it cannot commit an unsupported assertion as fact.

The architecture consequently permits deterministic reference execution, alternative solvers and later acceleration without changing the meaning of a historical observation or a journal entry. Performance work begins only after the same semantic cases pass through these interfaces.

Chapter 5

Historical compatibility and future simulation

5.1 Define what is observed before inverting it

Let D_k be the disclosures available under information cutoff k. Each numeric observation d_i has a context: entity or group perimeter, concept, measurement basis, dimensions, currency, period and precision. These contexts are part of the observation operator, not additional columns that can be discarded after extraction.

A candidate history h contains an opening economic and accounting state, events through the last historical date T, contract terms, accounting policies, consolidation relationships and latent quantities. Let M denote the declared model scope: its event templates, admissible rule versions and allowed unknowns. It excludes arbitrary events that the chosen model has no way to represent. The reporting operator $O_i(h)$ produces the quantity to which observation i refers, including its basis and perimeter.

Reported numbers are often rounded. If a number v_i is rounded to the nearest unit q_i, an illustrative admissible interval is $[v_i - q_i/2, v_i + q_i/2)$ under the corresponding tie convention. The actual rounding convention, scale and XBRL accuracy metadata govern the imported constraint. A disclosure with unspecified precision does not acquire a convenient tolerance silently. Write $\mathcal{I}_i$ for the justified observation interval, and $C_M(h)$ for the joint accounting, contractual and semantic constraints. Then the historical compatibility set is

$$\mathcal{H}_M(D_k) = \{h \in \mathcal{H}_M : C_M(h) \ \wedge \ O_i(h) \in \mathcal{I}_i \text{ for every applicable } i\}. \tag{5.1}$$

Qualitative assertions enter as reviewed logical conditions, rather than numeric intervals. Incompatible source contexts are never forced into the same equation.

Equation (5.1) distinguishes three outcomes. An empty set means that the observations and declared model are inconsistent. A singleton means that this model identifies a history at its chosen resolution. A larger set preserves unresolved alternatives. An empty set does not prove that the company's accounts are false: a wrong extraction, missing event type or incorrect perimeter can cause the inconsistency. A singleton does not prove that the model contains the real history.

5.2 Why published statements cannot identify all transactions

The simplest derivation uses fixed account mappings and a fixed basis. Let $z \in \mathbb{R}^n$ contain unknown historical movement amounts. For m disclosed aggregates, let $A \in \mathbb{R}^{m \times n}$ be the aggregation matrix and $y = Az \in \mathbb{R}^m$. If there is a nonzero vector v with $Av = 0$, then

$$A(z + v) = Az + Av = y.$$

Both movement vectors give the same reports whenever both satisfy the remaining constraints. Thus the feasible family is the affine set $z + \ker A$ intersected with those constraints. Accounting requirements can shrink this family, but do not generally eliminate it. For example, cash collections in a receivables bridge are constrained by

$$AR_T = AR_0 + \text{credit sales} - \text{collections} - \text{write-offs} + \text{other classified movements}.$$

If the note does not distinguish acquisitions, currency effects and write-offs, several collections paths may fit the same closing balance.

There is a second ambiguity beyond aggregation. In Equation (1.1), identical historical movements arise from different contractual reset terms. Even recovering z exactly would leave these terms unidentified. AccDSL therefore infers economically relevant states and contract terms alongside journal movements. A linear inverse solver over statement lines alone cannot solve this second problem.

5.3 The accounting state and its invariants

For one entity book with n_a accounts in one functional currency, let $b_t \in \mathbb{Q}^{n_a}$ be signed account balances, with debit amounts positive and credit amounts negative. Exact decimal amounts can be represented as rationals with a power of ten denominator. A journal batch j_t contains the signed functional-currency amounts posted by one atomic event or explicitly grouped event batch. Its balance condition is

$$\mathbf{1}^\mathsf{T} j_t = 0, \qquad b_t = b_{t^-} + j_t. \tag{5.2}$$

If $\mathbf{1}^\mathsf{T} b_0 = 0$, substitution gives $\mathbf{1}^\mathsf{T} b_t = \mathbf{1}^\mathsf{T} b_{t^-} + \mathbf{1}^\mathsf{T} j_t = 0$. Induction establishes conservation through every committed batch. Opening balances must therefore be supplied as a balanced, explained entry or a balanced imported state with unresolved composition explicitly identified.

This proof establishes a bookkeeping invariant. It does not establish that an amount belongs in the chosen account, that a contract exists, or that an accounting standard permits a treatment. Posting validation additionally checks account roles, ownership, currency conversions, applicability of rules and event preconditions.

The balance vector includes temporary revenue and expense accounts during the period. Period closing transfers their net balance to equity through balanced entries. A reporting function can include current profit in equity before closing, or use the closed book afterwards, but must not include it twice. Income and cash-flow reports use classified movements and exclude mechanical closing transfers from operating flows. Consequently statements are functions of balances, journals, contract state and reporting rules:

$$Y_{[u,v)} = R(b_{v^-}, J_{[u,v)}, X_{v^-}, \rho, [u, v)). \tag{5.3}$$

Here ρ specifies the reporting basis and perimeter, and X contains economic state beyond ledger balances. Cash-flow classification is part of R, not recoverable from the closing cash balance alone.

Each book has its own functional currency. Quantities such as shares, tonnes and foreign-currency units are typed separately and never added to this balance equation. A foreign-currency posting stores its original units and the functional-currency valuation with its rate source. Translation into a group's presentation currency and consolidation adjustments occur in a separate book under explicit rules.

5.4 Simulate events from economic state

For a chosen history h, the state at T is $X_T(h)$. It contains balances, open contracts, lots or cohorts, unpaid accruals, commitments and other quantities needed for future evolution. Let s identify a scenario definition, U_t^s its supplied economic drivers, a_t management's actions, and ε_t future innovations. A behavioural policy π chooses a_t using the permitted information $\mathcal{F}_t$:

$$a_t = \pi(X_t, U^s_{\leq t}, \mathcal{F}_t; \theta), \tag{5.4}$$

$$e_{t+1} = \mathcal{G}(X_t, a_t, U^s_{t:t+1}, \varepsilon_{t+1}; \theta), \tag{5.5}$$

$$(j_{t+1}, m_{t+1}) = \mathcal{A}(e_{t+1}, X_t, r_{t+1}), \tag{5.6}$$

$$X_{t+1} = \mathcal{T}(X_t, e_{t+1}, j_{t+1}, m_{t+1}). \tag{5.7}$$

The generator $\mathcal{G}$ emits events, such as a sale, rate reset, payment or default. The accounting function $\mathcal{A}$ applies rule package r and returns journal postings plus measurement annotations m. The transition $\mathcal{T}$ updates economic and ledger state atomically. A default can therefore change the loan's legal status, expected recoveries, recognised allowance and cash collections through related, but distinct, records.

This ordering is a reference abstraction. Within a date, event dependencies and phases specify the actual order. Expectations used to measure an allowance or provision must be calculated from the information permitted at that date. They cannot use the realised future path of the simulation. An inner expectation model may use quadrature, analytic formulas or nested simulation; its approximation and seed belong in the measurement annotation.

Some scenarios specify a deterministic macroeconomic path while leaving idiosyncratic defaults random. Others supply a joint stochastic driver model. A risk-neutral measure used to value a derivative is distinct from the real-world measure used to simulate business outcomes. Every stochastic adapter declares its measure and conditioning information. The accounting function receives the resulting valuation with that declaration.

5.5 Preserve sets and distributions separately

Let $K_{h,s}$ be the probability law of future reports produced by the forward model conditional on history h, scenario s and its declared behavioural assumptions. A deterministic simulation is the special case of a point mass. When no probabilities over historical explanations are justified, the natural answer is a family of laws,

$$\mathcal{K}_{M,s}(D_k) = \{K_{h,s} : h \in \mathcal{H}_M(D_k)\}. \tag{5.8}$$

The system can report each explanation, bounds on a deterministic outcome, or lower and upper event probabilities across this family. Those are different result types. A range of conditional means is not a confidence interval, and an envelope of conditional quantiles is not itself a single predictive distribution.

If a defensible probability measure $\mu(dh \mid D_k, M)$ supported on $\mathcal{H}_M(D_k)$ is supplied, histories and report outcomes must have specified measurable spaces. Assume that $K_{h,s}$ is a probability kernel: it is a probability measure on report outcomes for each h, and $h \mapsto K_{h,s}(B)$ is measurable for each measurable event B. The predictive law is then

$$\mathbb{P}(Y^s \in B \mid D_k, M) = \int K_{h,s}(B)\, \mu(dh \mid D_k, M). \tag{5.9}$$

For discrete explanations this is the usual weighted sum. For continuous unknowns, integration uses the declared measure over those unknowns, not a count of solver solutions. Bayesian weights require a prior and a likelihood; expert weights remain labelled as elicited. A likelihood must not treat a statement total and all its components as independent repeated evidence. Deterministic equality constraints require a reduced parameterisation or proper conditional-measure treatment, rather than an undocumented Gaussian penalty with near-zero variance.

Suppose the fixed-loan explanation receives weight w and the floating-loan explanation receives $1 - w$, solely as an illustration. Under the rate shock,

$$\mathbb{P}(I = 5) = w, \qquad \mathbb{P}(I = 7) = 1 - w, \qquad \mathbb{E}[I] = 7 - 2w.$$

The expected expense is useful but hides the two different outcomes. For a general scalar output with finite second moment, put $m(h) = \mathbb{E}[Y \mid h]$. Write $Y - \mathbb{E}[Y] = (Y - m(h)) + (m(h) - \mathbb{E}[Y])$. The cross term vanishes because $\mathbb{E}[Y - m(h) \mid h] = 0$. Taking expectations of the squared expression gives

$$\mathrm{Var}(Y) = \mathbb{E}_h[\mathrm{Var}(Y \mid h)] + \mathrm{Var}_h(\mathbb{E}[Y \mid h]). \tag{5.10}$$

The first term is uncertainty within a chosen explanation; the second captures differences between explanations. This decomposition requires the measure μ. Without it, the system reports the family in Equation (5.8), rather than manufacturing a variance.

Weights themselves can be uncertain. For an admissible class $\mathcal{W}$ of measures, AccDSL can compute

$$\underline{p}(B) = \inf_{\mu \in \mathcal{W}} \int K_{h,s}(B)\, d\mu(h), \qquad \overline{p}(B) = \sup_{\mu \in \mathcal{W}} \int K_{h,s}(B)\, d\mu(h).$$

If every measure supported on the feasible histories is allowed and point masses are included, these bounds reduce to the infimum and supremum of $K_{h,s}(B)$ over histories: averages cannot lie outside the range of their integrands, and point masses attain, or approximate, the extremes. Narrower weight restrictions require an explicit justification and a separate optimisation problem.

5.6 Resolution and computational limits

For a requested horizon H and scenario family $\mathcal{S}$, histories may be grouped only when they have the same conditional law for all requested outputs:

$$h_1 \sim_{H,\mathcal{S}} h_2 \quad \Longleftrightarrow \quad K_{h_1,s} = K_{h_2,s} \text{ on those outputs for every } s \in \mathcal{S}.$$

This definition explains why individual sales can sometimes be replaced by cohorts, while fixed and floating loans usually cannot be merged for an interest-rate stress. Exact equivalence can be difficult to prove. A grouping justified only by approximate numerical agreement carries an error criterion and is not labelled exact. Source provenance remains available even when execution shares a common state.

Bounds require comparable honesty. A linear program can provide certified extrema for a supported linear model. A set of feasible sampled histories provides observed extrema: its

minimum is no smaller than the true infimum, and its maximum is no larger than the true supremum. It therefore supplies an inner exploration of possibilities, not a certified outer bound. Interval methods may provide outer bounds that no single history attains. Every result records which of these meanings applies.

The initial implementation offers exact bounded choices and linear constraints, symbolic continuous families such as $\alpha \in [0, 1]$, and explicit scenario execution at selected feasible states. General nonlinear partial identification, automatic model discovery and exhaustive enumeration of all economic histories are research extensions, not promises made by the core language.

Chapter 6

The accounting language

6.1 Scope of the first language

AccDSL Core 0 describes entities and books, accounts, observations, shared unknowns, constraints, alternative interpretations, contract instances, accounting rules and reporting mappings. It has pure expressions and a restricted posting effect. Economic forecasting algorithms can live in typed plugins; unrestricted numerical code does not need to be reimplemented in the DSL. The grammar and interface details in Appendix A are normative for the first parser.

The compiler has six ordered stages: parsing with source locations; name and module resolution; type and temporal-context checking; lowering to the IR; constraint and rule-applicability analysis; and execution-plan construction. The compiler never silently supplies an economic assumption to make a program compile. A program can be valid as a historical representation while requiring additional assumptions before a particular simulation can run.

The language's meaning is defined by its IR and evaluator, rather than by the surface spelling alone. Formatting changes cannot change a model's semantic hash. A changed source span or reviewer decision changes evidence provenance even if the numerical model remains the same. The system therefore maintains separate evidence, semantic-model and execution identifiers.

6.2 Types that protect meaning

Type	Meaning and checks
`Money<C>`	Exact amount in currency C. Addition requires identical currency; conversion requires a dated rate and policy.
`Quantity<U>`	Units of shares, goods or other commodities. Quantities cannot be added to money or used as a functional-currency posting.
`Rate<B>` and `YearFraction`	Rate with basis B, plus a separately computed accrual fraction. Simple and compound accrual functions are distinct.
`Instant`, `Period`, `Calendar`	Instant observations and flows over intervals; interval operations preserve calendar conventions.
`Account<K,Book>`	Account role K and owning book. Cross-book effects require separate balanced batches linked by an economic event.

Type	Meaning and checks
`Observation<T>`	Source assertion with contextual and precision metadata; accessing its value retains its constraint and provenance.
`Symbol<T,D>`	Shared unknown with domain D; repeated references denote the same value in a history.
`Choice<E>`	A finite structural choice over enumeration E. No probability is implied.
`Distribution<T>`	Explicit probability measure with parameter, conditioning and measure metadata. It is not a Symbol or a Choice.
`SourceRef`, `RuleRef`, `ModelRef`	Versioned evidence or implementation references; unresolved references prevent concrete execution.
`Measurement<T>`	Value plus valuation date, basis, method, assumptions and numerical status. Recognition remains a rule decision.

Core expressions use exact integers and rationals, Boolean logic, enumerations, dates, records, lists, comparisons and registered pure functions. A decimal literal is parsed exactly, not first converted through binary floating point. Numerical plugins can return approximate measurements, but must declare their error or numerical status and the rounding used when posting money.

Missingness is a tagged type with distinct values: `notDisclosed`, `notApplicable`, `unreadable`, `withheld`, and `unresolvedMeaning`. Zero is an ordinary known number. A missing datum can be replaced by a declared Symbol or assumption in a model branch; the source observation stays missing. An unknown is not necessarily random, and an estimate is not necessarily missing.

Addition of monetary observations also requires compatible entity, perimeter, time, dimensions and basis. These checks operate on contextual expressions in addition to the currency type. An analyst can deliberately aggregate dimensions or transform a basis using an explicit function. For example, `sumOver(segment, ...)` records the dimension being removed and requires the segment partition to be declared; it does not assume every disclosed segment total is additive.

6.3 A small model with a continuous interpretation family

The following source uses a synthetic observation whose full context is stored in the evidence registry. The example imports the demonstration debt package, which uses one-year simple accrual, no fees and immediate rate resets. It is not an IFRS rule package.

```
module BorrowingExample version "1";
use DemoDebt version "1";
source note7 = external("synthetic-note-7");
entity Acme functional USD;
book Local entity Acme basis DemoDebt version "1";
account Cash : Asset book Local;
account Debt : Liability book Local;
account InterestExpense : Expense book Local;

observation debt : Money<USD> instant @2025-12-31
  = 100 USD from note7;
symbol alpha : Real in [0, 1]
  because "The fixed-rate proportion is not disclosed";
```

The remaining declarations use this same unknown fraction in both loan components. A change to the fraction therefore changes the whole debt mix consistently.

```
let fixedPrincipal = alpha * value(debt);
let floatingPrincipal = (1 - alpha) * value(debt);
constraint split:
  fixedPrincipal + floatingPrincipal == value(debt);

contract loans : DebtMix book Local {
  principal = value(debt);
  fixed_fraction = alpha;
  fixed_rate = 0.05 per year;
  initial_floating_rate = 0.05 per year;
}
alternative allFixed { assume alpha == 1; }
alternative allFloating { assume alpha == 0; }
scenario rateShock {
  set market_rate = constant(0.07 per year);
}
```

The parent model retains every $\alpha \in [0, 1]$. The two named alternatives are useful conditional views of that family, not a declaration that the endpoints exhaust it. Requesting the parent family's exact bounds produces $[5, 7]$ for the next year's expense in this simple model. Requesting a conditional run of `allFixed` supplies $\alpha = 1$. Requesting a weighted mixture requires a separate weight model; the two labels do not create equal weights.

An observation declaration is a reference to evidence plus a proposed typed reading. The compiler checks that the registry context agrees with the declared period and type. If the evidence is rounded, `value(debt)` denotes a shared constrained value rather than a silently exact 100. This synthetic registry explicitly declares an exact value, which keeps the illustration deterministic.

6.4 Posting rules and explicit exceptions

An event type declares the information needed by a rule. The example below assumes an event representing interest accrued and paid on the same date. Separate accrual and settlement events are needed when those dates differ.

```
event InterestPaid(principal: Money<USD>,
                   rate: Rate<annual>,
                   fraction: YearFraction) book Local;

rule recogniseInterest for InterestPaid as e
when e.principal >= 0 USD {
  let charge = roundMoney(e.principal * e.rate * e.fraction,
                          currencyScale(USD), "half_even");
  require charge >= 0 USD;
  post {
    debit InterestExpense charge;
    credit Cash charge;
  }
}
```

The rule cannot perform file I/O, mutate another entity or sample a random number. It returns a proposed batch. The runtime verifies balance and all preconditions before committing

state. Negative interest requires an explicit supported rule; it is not mishandled by this intentionally restricted example. Rounding is a declared demonstration convention, not a universal accounting default.

Rules can declare `overrides ruleName`. The resulting override graph must be acyclic. For a concrete event, the evaluator determines all applicable rules and removes any that are overridden by another applicable rule. Exactly one maximal rule must remain for a required recognition operation. An empty set of applicable rules produces `UnsupportedTreatment`; more than one maximal rule produces `RuleConflict`. File order and a hidden numeric priority never break ties.

If a guard depends on a symbolic value, the solver partitions the current feasible family into the guard and its negation, subject to the existing constraints. This is a logical partition, not a random draw. An incomplete solver returns `UnknownApplicability`, not a guessed rule. Statically provable disjointness or total coverage can be cached; otherwise the runtime performs the same checks on every concrete event.

6.5 Alternative readings, assumptions and updates

An alternative adds constraints and possibly a supported semantic mapping to its parent model. Its record contains a rationale, supporting spans, opposing evidence, author, creation time and scope. Child alternatives inherit all parent constraints; they cannot quietly discard an inconvenient disclosure. A deliberate source rejection is a separate review decision with an explicit reason and a new evidence view.

Two independent alternatives may coexist, while two mutually exclusive readings of the same text require an explicit choice group. The representation stores these relationships in a constraint graph. It does not take a Cartesian product blindly. For example, interpreting an instrument as a liability affects both its balance-sheet account and the treatment of its periodic payments; those choices must be linked through one shared treatment variable.

New evidence narrows an existing family by conjunction when it concerns the same definitions and reporting basis. If it changes the basis or corrects an earlier assertion, the system creates a revised evidence view and records the relationship. Posterior reweighting, when enabled, is a separate operation with likelihood provenance. A branch that becomes inconsistent is retained as historically considered and rejected, with the evidence that rejected it.

6.6 Modules and stable intermediate representation

A package identifies its namespace, semantic version, dependencies, effective dates, jurisdiction, supported entity classes, recognition operations and measurement policies. Standard packages define allowable treatments; industry packages add economic objects and specialised rules; company packages map source concepts and declare evidenced elections. A company extension cannot override a mandatory standard rule merely because its package is loaded last. An override is valid only at a declared extension point and within the selected standard's policy domain.

The IR has separate node classes for evidence, contextual observations, expressions, domains, constraints, alternatives, contracts, economic events, measurement results, rules, journal batches, report mappings and runs. Each node has a stable identifier and explicit input references. No expression stores executable host-language text. Plugins are referenced by versioned identifiers with typed signatures. This permits a second implementation to execute

the same supported program without evaluating arbitrary embedded code.

The IR serialisation version is independent of the DSL version and of any accounting-standard edition. Migrations preserve the original payload and record changed meanings. A syntax migration can be automatic when semantics are unchanged. A changed recognition policy requires a new model version, even if the source file still parses.

6.7 Errors are part of the interface

The compiler and runtime return typed diagnostics with a source span, affected entity and period, dependency chain and suggested local action. The principal diagnostic categories have distinct recovery paths:

Diagnostic	Meaning and next action
`TypeMismatch`, `ContextMismatch`	Resolve incompatible units, periods, entities or measurement bases; do not coerce implicitly.
`MissingEvidence`	Supply a source or label a model input as an assumption.
`InfeasibleHistory`	Inspect the conflicting evidence and model constraints; preserve the rejected candidate.
`RuleConflict`, `UnknownApplicability`	Resolve rule scope or preserve a justified conditional partition.
`UnsupportedTreatment`	Add a reviewed capability or leave the affected output incomplete.
`UnbalancedBatch`	Reject the atomic update and diagnose the posting rule.
`SolverIncomplete`, `NumericalFailure`	Preserve the target and inputs; repair or change the computational method.
`InsufficientSimulationState`	Add declared conditional assumptions or request a family of results.

Warnings can describe low extraction confidence or an approximate economic model, but they do not replace these errors. A required unknown input prevents a concrete unconditional result. The analyst can add a declared conditional assumption, request bounds, or leave that output incomplete. Every path has a different, explicit result type; there is no generic fallback to zero.

Chapter 7

Reconstruction and execution procedures

7.1 Ingest, reconcile and construct histories

The historical procedure starts with a filing manifest. It records documents, reporting dates, publication dates, revisions and expected statements and notes. Tagged filings are parsed through an XBRL adapter; PDF and HTML sources are segmented into text, tables, captions and references. The original bytes and extraction output remain immutable. Re-running an improved extractor creates a new extraction version with links to the same source.

For each table, the importer first resolves headers, units, signs, periods and dimensions. It then proposes concept mappings. A deterministic validator checks subtotal arithmetic, duplicate facts, comparative consistency and cross-note references. Every proposed repair creates a new reading. The system may suggest that an OCR value is 10 rather than 70, but only a reviewed correction or an explicit alternative allows that reading into a validated model.

Reconciliation proceeds from local totals to movement schedules. A debt bridge may contain opening debt, issuance, repayment, acquired debt, interest accretion, currency effects, fair-value changes and closing debt. Every component receives a movement type. An unexplained residual is stored as a residual with a source and period; it is not automatically classified as repayment or issuance. A model can introduce an unknown decomposition of that residual, subject to evidence and economic constraints.

The reconstruction engine then performs the following finite procedure within a declared model scope:

1. Build shared variables for observations, opening states, contract terms and supported movement amounts. Attach original rounding domains.
2. Add identities, roll-forwards, account restrictions, contractual relationships and cross-period links. Add each semantic reading as a condition on the same joint model.
3. Propagate types, units, intervals and simple equalities. Reject context mismatches before invoking a numerical solver.
4. Ask the solver for feasibility. On infeasibility, return a sufficient conflicting subset of constraints, its source spans and model assumptions. A solver's conflict core need not be the smallest possible explanation.
5. Retain finite alternatives and continuous solution families symbolically. Materialise ex-

ample histories only when needed for review or execution.

6. Check whether each requested simulation has sufficient economic state. Missing maturities, reset dates or cohort composition become declared assumptions or explicit unresolved inputs.

Unrestricted journal enumeration would be unbounded: arbitrary offsetting transactions can be inserted without changing reports. The model therefore declares allowed event types, time resolution, relevant accounts, lot/cohort resolution and limits on latent contract instances. These limits describe the represented family; they are not evidence that all real possibilities were searched. The compiler canonicalises syntactic duplicates and equivalences established by its supported rewrite rules. It does not decide arbitrary economic equivalence. Probability measures attach to declared economic hypotheses or parameters, never to a count of encodings; adding a duplicate representation therefore cannot create extra probability mass or coverage.

The first implementation uses symbolic linear bridges and finite structural choices. Nonlinear economic models can be evaluated on feasible states through plugins. A solver timeout preserves the unresolved family and returns `SolverIncomplete`; it does not prune the family as impossible. Search budgets belong to run configuration and are recorded with the results.

7.2 Condition a run without rewriting history

A run selects an evidence view and cutoff, a historical family, a forecast starting date, scenario definitions, accounting packages, management policies, output queries and numerical settings. A run may use a concrete history, an exact symbolic family, a finite set of conditional histories, or weighted draws from a declared distribution. The output records which mode was used.

Suppose no debt maturity is disclosed. An analyst may run a conditional case that assumes all debt matures after the forecast horizon. The run is valid under that assumption even though the historical representation remains incomplete. The interface must display the assumption next to the result and provide sensitivity cases when the assumption is material. It must not label the assumed maturity as a discovered historical fact.

Management behaviour normally uses only information available at the decision date. A perfect-foresight stress case is a separate, explicitly labelled policy. Scenarios should share the same underlying innovations when estimating paired differences where appropriate, with stable stream identifiers so branch order does not change the draws. Common random numbers improve the comparability of paired simulations; they do not imply that every branch has the same economic event counts or shock distribution.

7.3 Schedule and commit events

Each event carries an economic occurrence time, accrual interval when relevant, settlement time, entity, contract or cohort identifier, currency, inputs and provenance. Its ordering key consists of date, declared phase, dependency edges and a deterministic tie identifier. The reference phases are: opening and scheduled contractual changes; market observations and rate resets; business events and decisions; accrual and measurement; settlement; period closing and reporting. A contract can require a more specific dependency. These phases are an engineering starting point, not a universal description of market timing.

Two events may share a tie order only if their order is either specified or irrelevant to the result. If payment before remeasurement and payment after remeasurement differ, an unresolved

ordering is a model ambiguity or an error. The engine does not conceal it behind alphabetical event IDs.

```
execute_event(event, state, run_context):
    verify prerequisites and information availability
    resolve applicable measurement and recognition rules
    compute proposed economic changes and measurements
    compute proposed journal batch(es)
    validate types, contexts, balance, constraints and idempotency
    atomically commit state, journals and explanation edges
    return new state, emitted dependent events, diagnostics
```

The atomic unit includes the economic state and its accounting effects. If posting fails, neither state changes. A repeated event ID with identical inputs returns the existing result; the same ID with different inputs is a conflict. Consolidation runs after entity books reach the required date and stores its adjustments separately. A failure in one scenario branch cannot mutate another branch's state.

7.4 Funding, tax and other simultaneous relations

Some financial models have circular dependencies. Interest affects taxable profit; tax and investment affect funding needs; funding affects interest. The reference executor first tries to resolve these through explicit timing. For example, beginning-of-period debt can determine current interest, with new borrowing affecting the next accrual period. That is acceptable only when the model's timing assumption is stated and appropriate.

Where the economic model genuinely requires simultaneous decisions, the dependency graph identifies the strongly connected component and delegates it to a declared equation or optimisation solver. The solver must return a solution, residuals, admissibility checks and uniqueness or multiplicity status. A contraction iteration requires a justified convergence condition; an arbitrary fixed iteration count is not a mathematical solution. Multiple admissible equilibria become alternatives when material. No solution produces a model outcome such as financing failure, or an explicit numerical failure, depending on the cause.

Funding policy must include available facilities, limits, covenants, collateral and allowed capital actions. The program cannot issue unlimited debt to keep cash positive. An inability to meet payments may be a legitimate default outcome. Default ends or changes contractual and accounting processes under the relevant event rules; it is not quietly discarded as a bad simulation draw.

7.5 Worked example: identical history, different future journals

Return to the company in Chapter 1. Immediately after the historical interest payment and closing, cash is 95, debt is 100 and equity is -5. There are no other assets, liabilities or transactions. Next year's expense is 5 in the all-fixed branch and 7 in the all-floating branch under the rate shock. The generated entries and closing statements are:

Output	All fixed	All floating
Debit interest expense / credit cash	5	7
Closing cash	90	88
Closing debt	100	100
Current-year profit	−5	−7
Closing equity after profit transfer	−10	−12
Cash change	−5	−7

The balance-sheet identity holds in each branch: $90 = 100 - 10$ and $88 = 100 - 12$. The demonstration reporting profile classifies interest paid as operating cash flow. A different permitted reporting profile changes that classification, while preserving the cash movement and source event. This example proves the intended distinction and its arithmetic; it does not estimate the probability that either debt structure describes a real company.

7.6 Worked example: a bond has several relevant amounts

A two-period zero-coupon bond raises cash of 100 and promises 110.25 at maturity. Assume a 5% effective rate per period, no transaction costs, no default, and amortised-cost treatment under a demonstration rule. The effective-interest schedule follows directly from $100(1.05)^2 = 110.25$:

Date	Opening liability	Interest accretion	Closing liability
Issue	0	0	100
End of period 1	100	5	105
End of period 2, before payment	105	5.25	110.25
Redemption	110.25	0	0

Issuance debits cash and credits the liability by 100. Each accretion debits interest expense and credits the liability. Redemption debits the liability and credits cash by 110.25. Legal face amount, carrying amount and market fair value remain separate fields. A market-rate shock may change fair value without changing this contractual payment schedule or this demonstration amortised-cost path.

Cash-flow reporting uses the actual payment event and its economic components; the annual accretion is non-cash. The applicable reporting package decides how to classify the maturity payment, rather than inferring it solely from the liability debit. Fees, modifications, early redemption, embedded options, hedges and foreign currency require additional event and measurement rules.

Ordinary equity issuance similarly separates cash, share quantity and legal capital. Ten new shares at 12, with par value 1, produce cash of 120, share capital of 10 and premium of 110 under a simple par-value profile. Shares outstanding increase by ten units. Direct issuance costs of 3 reduce the specified equity account and cash under the chosen illustrative treatment. The accounting rule and the cost's eligibility must be explicit; this example does not prescribe every jurisdiction's treatment.

7.7 Reporting and explanations

The reporting engine produces the balance sheet, income statement, comprehensive income, cash-flow statement, changes in equity and note schedules when the necessary modules are

present. Each output carries its basis, perimeter, period, scenario and interpretation. A comparative table cannot combine an original prior-year figure and a restated current-year comparative without displaying that choice.

An explanation is a dependency graph with typed edges: extracted from, interpreted as, assumed by, constrained by, generated by, measured by, posted by, aggregated into and supersedes. A cash-flow line must lead back to payment events; a fair-value movement must lead back to valuation inputs; a consolidated amount must show eliminations. A provenance path establishes how a result was obtained. It does not by itself establish that the underlying economic model is correct.

Chapter 8

Standards, industries and financing instruments

8.1 A common core with distinct accounting books

The common core governs evidence, time, money, events, journals and provenance. Reporting standards govern recognition, measurement and presentation within that core. The design preserves three levels of meaning: economic terms such as a promise to pay cash; an accounting treatment under a particular standard; and a company's reported label or extension. These levels are connected by explicit rules and mappings, rather than collapsed into one universal chart of accounts.

A rule package includes a standards register with jurisdiction, issuer class, effective dates, transition provisions, permitted elections, paragraph references, implementation version and tests. The package selector uses the entity, reporting basis, period and adopted elections. It must distinguish a standard's mandatory effective date from permitted early adoption. For example, the IFRS Foundation's IFRS 18 overview states an effective date of 1 January 2027 with earlier application permitted (IFRS Foundation, 2026c). A 2026 historical report cannot be assigned that presentation regime solely because the latest installed package includes it.

IFRS, US GAAP and ASBE packages share economic contracts and events where appropriate, but produce separate accounting books. A reconciliation between books consists of explained adjustments with origins in differing treatments, not a renaming of accounts. The first implementation can preserve all three source bases even while only a selected subset has executable rules. Such a source is represented as observed information with unsupported simulation treatments, rather than being silently converted to the supported basis.

The official IFRS overviews for IFRS 9, IAS 32 and IFRS 17 establish the relevant scope of financial-instrument measurement, presentation and insurance contracts (IFRS Foundation, 2026d,a,b). They are orientation sources, not substitutes for reviewed paragraph-level implementations. Public retrieval of the selected Chinese Ministry of Finance page and SEC API documentation failed during this campaign; the source register records these access gaps. ASBE and US GAAP semantic modules therefore have explicit primary-text acquisition and accounting-review tasks before they can be marked executable and compliant.

8.2 Non-financial companies

The initial industry package covers sales, collections, purchases, payments, inventory movements, payroll, capital expenditure, depreciation and basic financing. Revenue recognition is a separate rule from invoicing and cash collection. A service contract may generate a performance event before an invoice, creating a different asset from an unconditional receivable. Customer advances create obligations before revenue. Returns, rebates and variable consideration need separate assumptions and subsequent adjustments.

Inventory state includes quantity, cost, location, ownership and lot or cohort. A costing method is selected within the reporting package's permissible domain. An aggregate stock can support a simple model only if its approximation is declared. A commodity-price shock can expose differences between old and new inventory that an average value suppresses. Fixed-asset state likewise requires acquisition dates, components, useful lives, residual values, depreciation method, tax basis and impairment history when those affect the requested forecast.

Industry extensions add economic mechanisms without bypassing the accounting core. Construction requires performance and completion states; extractive industries require reserves, depletion and restoration obligations; utilities may require rate-regulation treatment; software businesses may require contract liabilities and development-cost decisions. The module manifest states what is represented, what is executable, and what remains aggregated.

8.3 Debt and hybrid financing

The contract layer for a debt instrument stores issuer and holder perspective, currency, issue and settlement dates, issue proceeds and legal face amount. Its payment terms include coupon/index/spread, reset and fixing dates, day-count convention, payment calendar, amortisation schedule and maturity. Its rights and protections include seniority, collateral, covenants, options, conversion terms, guarantees and linked hedges. Missing terms are explicit unknowns. A single carrying amount cannot stand in for this record.

The accounting layer stores the classification decision and its evidence, initial measurement, carrying amount, accrued interest, effective-rate method, unamortised fees or discounts, fair-value disclosures, impairment or credit adjustments where applicable, and presentation mappings. Contractual interest, effective-interest expense, realised cash payments and fair-value changes are separate quantities. The zero-coupon example in Chapter 7 provides a minimal conformance case for this separation.

The lifecycle must support issuance, recognition, accrual, reset, coupon, principal repayment, modification, derecognition, default, recovery and conversion. A modification event first determines which accounting operation the selected standard requires; it does not always mean issuing a new loan. An embedded option can affect valuation and classification without being exercised. A derivative linked as a hedge requires both the economic contract and any applicable accounting designation; economic risk reduction alone does not establish hedge-accounting eligibility.

Hybrid instruments make interpretation particularly consequential. IAS 32's overview discusses liability/equity classification and the separation of a compound instrument (IFRS Foundation, 2026a). The executable classification must consider the full terms and the selected standard version. US GAAP or ASBE treatment must be implemented under its own rules rather than inferred from an IFRS classification. Where the terms are not disclosed sufficiently, alternative classifications can coexist only if each is actually compatible with the known terms and applicable rules.

8.4 Equity and ownership

Equity has both monetary accounts and security quantities. The security register records share class, issued and outstanding quantities, treasury holdings, par or stated value where applicable, voting rights, dividend rights, redemption features, options and conversion relationships. Monetary equity accounts include contributed capital, reserves, retained earnings and other components required by the selected basis. These records evolve together through issuance, repurchase, cancellation, split, dividend and conversion events.

A stock split changes quantities and per-share measures without necessarily changing aggregate monetary equity. A cash dividend affects distributions and cash rather than current operating expense under the corresponding ordinary equity treatment. Repurchases and employee share awards require their own rules. Earnings per share uses a dated quantity schedule and rules for potential ordinary shares; simply dividing profit by the closing share count is insufficient when shares change during the period.

Funding simulations must connect the quantity register to market assumptions. An equity issue at a scenario-dependent price changes cash raised, shares and dilution. The model must state whether it targets a fixed cash amount, a fixed share count or a capital ratio. These are different management decisions even when the historical statement would report the same aggregate proceeds.

8.5 Banks: accounting capital is not regulatory capital

A bank package needs instrument and cohort states for loans, securities, deposits, wholesale funding, repos, derivatives, collateral and commitments. For interest-rate analysis, currency and repricing buckets matter alongside remaining maturity. For credit analysis, exposure, default state, loss severity, recoveries, guarantees and their dependencies matter. For liquidity analysis, deposit behaviour, collateral eligibility, encumbrance and payment timing matter. These states can be represented at an explicitly chosen aggregation level; the public record rarely identifies every contract.

Loan origination can debit loans and credit a customer deposit in the bank's book. Subsequent transfer to another bank produces settlement flows. Treating all loan growth as an immediate cash outflow would therefore encode a specific funding assumption, not a universal banking identity. A system-wide model must also link counterparties and settlement assets; single-company simulation records external counterparties without claiming to close the whole economy.

An allowance calculation is a dated measurement based on forecast information. For a single horizon, let EAD be a fixed exposure at default, PD the probability of default, and LGD the mean loss fraction conditional on default. Ignoring discounting, the illustrative expected loss in currency units is

$$L = EAD \times PD \times LGD.$$

With exposure 100, default probability 0.02 and loss given default 0.4, it produces 0.8. Changing those inputs to 0.04 and 0.5 produces 2, so the allowance increase is 1.2 if there are no other movements. This calculation is a demonstration model. A standard-specific expected-credit-loss implementation must determine horizon, staging or other classification, discounting, forward-looking information, write-offs and purchased-credit-impaired or other special cases. The simple formula is not an IFRS 9 or US GAAP compliance implementation.

Regulatory capital is computed in a separate prudential layer with its own perimeter, eligibility, deductions, risk weights and transition provisions. Accounting equity feeds a reconciliation to

capital; it is not automatically Common Equity Tier 1. Liquidity and capital constraints influence the management policy, which in turn generates asset sales, funding or capital events. A breached ratio can be an economically meaningful scenario outcome. No hidden capital injection is permitted to repair it.

8.6 Insurance: cohorts and future service

An insurer requires policy cohorts or groups with issue dates, coverage, premiums, benefits, expenses, lapses, claims development, guarantees and reinsurance relationships. Asset portfolios need their own contracts and valuations. The liability state must distinguish expected future cash flows, discounting, risk adjustments or margins, unearned profit measures where applicable, incurred claims and movements between opening and closing states. The IFRS 17 overview's distinction between measuring future cash flows and recognising profit as service is provided motivates this separation (IFRS Foundation, 2026b).

The module API accepts policy/cohort state, economic assumptions, non-economic assumptions and information date. It returns a measurement vector, movement attribution and proposed recognition events. Experience differences, changes in assumptions, finance effects and service effects remain separately identifiable. Reinsurance held has its own record and measurement; it is not an automatic percentage offset to every insurance liability.

Scenarios can alter discount curves, mortality, morbidity, lapse rates, inflation, catastrophe losses and policyholder behaviour. Their joint dependence and cohort exposure must be declared. A portfolio total can match history while hiding very different guarantee and duration profiles. This makes alternative economic histories especially valuable, but also limits what public-data reconstruction can identify. A full insurance package needs actuarial and accounting review beyond the core release.

8.7 Capability contracts for sector packages

Package	Required state and operations	Minimum acceptance evidence
Core corporate	Working-capital bridges, lots/assets, ordinary debt and equity, period closing and cash-flow mappings	Exact synthetic cases, reviewed public filing reconstruction, all statement reconciliations.
Financial instruments	Terms, calendars, resets, options, measurement categories, modifications and hedges	Contract schedule comparisons, valuation checks, paragraph-based posting cases and transition cases.
Banking	Credit cohorts, deposits, collateral, liquidity, regulatory capital, management actions	Allowance roll-forwards, repricing and runoff stresses, capital reconciliation and default/funding-failure cases.
Insurance	Policy groups, future cash flows, service and finance components, reinsurance and asset interaction	Cohort movement reconciliation, assumption change cases, service recognition and reviewed actuarial calculations.
Multi-basis reporting	Versioned IFRS/US GAAP/ASBE rules, entity elections and transition treatment	Matched economic-event cases with explained basis differences and independent accounting review.

Every package declares capabilities at operation level. “Supports bonds” is too vague; “supports fixed-rate, single-currency, non-callable amortised-cost issuance and redemption under this reviewed rule set” is implementable and testable. Unimplemented operations retain their source and contract representation while producing an explicit capability error when execution requires them.

Chapter 9

Implementation contracts

9.1 Reference implementation and component boundaries

The initial implementation uses Python for orchestration and a deterministic reference evaluator, exact decimal/rational arithmetic for booked amounts, SQLite for transactional metadata and versioned records, and columnar files for large scenario outputs. DuckDB is an optional analytical reader. These are convenience choices justified by inspectability and modest initial scale; they do not establish production performance. The IR and conformance tests allow a later compiled executor without changing accounting meaning.

```
accdsl/
  evidence/       documents, spans, filing contexts, extraction versions
  model/          typed IR, entities, books, symbols, contracts
  language/       grammar, parser, resolver, type checker, lowering
  constraints/    propagation, solver adapters, bounds and conflict cores
  accounting/     rule selection, posting validation, closing, consolidation
  economics/      contract and behavioural plugin interfaces
  execution/      scheduler, branch state, stochastic streams, run records
  reporting/      statements, notes, movement schedules, explanations
  storage/        versioned records, migrations, result readers
  interfaces/     Python API, CLI, structured import/export
  packages/       reviewed demonstration and accounting modules
```

The first service boundary is local and synchronous. A job abstraction can be added for extraction and simulation without requiring distributed execution. The model compiler returns an immutable execution plan; the executor receives that plan and run inputs. Neither should retrieve live market data implicitly. Input datasets and rule packages are selected and versioned before execution.

9.2 Persistent records

Relation	Key information and constraints
`document`	Hash, original path/URI, filing ID, entity, publication and availability times, media type, revision links. Original bytes are retained.
`source_span`	Document hash, page/element locator, bounding box, text/table representation, extraction run ID.

Relation	Key information and constraints
`observation`	Context ID, value or missingness tag, precision domain, source-span IDs, semantic-reading ID, review status.
`context`	Entity/perimeter, concept, basis, currency/unit, instant or interval, dimensions and taxonomy version. Contexts are immutable.
`assertion_version`	Logical assertion ID, payload hash, economic validity, ingestion time, public availability, superseded version and reason.
`model_node`	IR node ID, type, serialised payload, dependency IDs, package/model version; no executable arbitrary text.
`constraint`	Expression, scope, source/assumption references, supported solver fragment, applicable alternative condition.
`alternative`	Parent model, added conditions, rationale, evidence, coverage declaration and rejection history.
`economic_event`	Stable event ID, contract/cohort, dates, quantities, dependencies, scenario and branch.
`journal_batch`	Event ID, book/basis, posting date, rule version, balanced postings, measurement IDs and atomic commit status.
`run`	Evidence and model hashes, cutoff, scenarios, branch selection, weight model, solver/numerical settings, seeds and environment.
`report_cell`	Run, branch/scenario, context, amount or result family, dependency references and completeness status.

Persistent identifiers are opaque; display labels can change without changing identity. References use foreign keys or validated content-addressed links. The full logical key of a reported fact includes its dimensions, not merely company, label and year. Duplicate assertions are stored as distinct source claims until a documented duplicate-resolution operation relates them.

Version records are append-only. A derived transaction-time interval can be closed by the next version event without deleting the original payload. Validity intervals can overlap when assertions conflict. Such overlap is diagnosed and resolved by the evidence view, not prohibited by a database constraint that would erase one assertion. Scenario states use structural sharing of immutable historical nodes and branch-local changes.

Large simulation output is indexed by run, explanation, parameter draw, scenario, innovation path, entity, book, date and account. Reducing this to means across all dimensions would destroy the product's central feature. Online summaries may compress draws within a declared distribution, while retaining conditional summaries and representative replay identifiers. A retention policy that removes individual traces must state which explanations can still be reconstructed.

9.3 Public API and result types

The Python and command-line interfaces expose the same operations:

```
ingest(manifest, extractor_config) -> EvidenceVersion
compile(source_or_ir, evidence_view, package_lock) -> ModelPlan | Diagnostics
reconcile(plan, cutoff, solver_config) -> HistoryFamily | Diagnostics
condition(family, assumptions) -> ConditionalFamily | Infeasible
simulate(family, scenario_set, run_config) -> RunResult
render(run_id, report_request) -> ReportBundle
explain(node_id, evidence_view_or_run_id) -> ExplanationGraph
compare(run_a, run_b, alignment_spec) -> Comparison
```

`HistoryFamily` contains a model-scope declaration, shared variables, constraints, finite choices, feasibility status and solver-completeness status. It may contain witness histories without claiming that those witnesses enumerate the family. `RunResult` is a tagged union of concrete outputs, conditional samples, certified bounds, explored ranges and unresolved outputs. Probability-bearing results reference a probability-model record. Results without that reference cannot be presented as probabilities.

Comparison requires an alignment specification: entity/perimeter, dates, reporting basis, scenarios and shared innovations where applicable. Comparing IFRS and US GAAP books holds economic events fixed and changes accounting rules. Comparing management responses holds the specified scenario and initial information fixed and changes the behavioural model. These are different questions and must not be combined into an unexplained difference.

9.4 Plugin interfaces

Every plugin declares a version, supported types, input information dates, determinism, numerical measure and capabilities. A contract plugin implements `schedule`, `observe`, `transition` and `payoff`; it may return future conditional events. A behaviour plugin implements `decide` and declares which state and scenario information it uses. A measurement plugin implements `measure` and returns an amount or vector with a basis and numerical status. A reporting plugin maps classified journals and balances to output contexts.

The plugin response must include all inputs that materially affect results. An external calculator's global evaluation date, market-curve cache, calendar version or random seed cannot remain invisible. For example, QuantLib's floating coupon interface makes fixing-date conventions and pricing objects relevant inputs; the adapter records them rather than relying on process-global state. ACTUS risk-factor observations receive explicit dates and identifiers.

The constraint interface accepts a typed formula and a requested operation: feasibility, witness, enumeration of declared finite choices, or bound. It returns `sat`, `unsat` or `unknown`, plus evidence appropriate to the operation. Timeouts and unsupported theories return `unknown`. A witness proves existence within the formula, not completeness of a recovered historical family.

9.5 Human review and model-assisted extraction

The review interface should put the source passage beside the proposed observation, context and reconciliation impact. An analyst can accept a mapping, add an alternative, reject an extraction, define a symbolic unknown or request more evidence. These actions create structured changes with reasons. The useful unit of review is a coherent disclosure and its links, not an isolated number detached from its note.

A language model can identify candidate tables, draft concept mappings, propose contract terms and suggest alternative readings. Its output must reference source spans and a schema. Deterministic checks reject fabricated source references, invalid units, inconsistent periods and unbalanced postings. A model-generated interpretation that passes those checks remains a proposal until its evidentiary status is established. Token probabilities and self-reported confidence are never automatically used as weights over economic histories.

For routine local research use, a lightweight interface and immutable source versions are sufficient. Multi-user approval hierarchies, cryptographic attestations and deployment security systems are outside this initial design. They can be added for a concrete operational requirement without changing the scientific representation.

9.6 Failure recovery and reproducibility

Extraction jobs and simulation jobs write explicit progress records and atomic results. Restarting a failed document conversion reuses its document hash and creates either the same idempotent extraction result or a new version under changed settings. Restarting a simulation resumes only from a committed state whose model and input hashes match the run. Numerical failures retain the inputs needed for a focused diagnostic.

A run manifest records code revision when available, package lock, source hashes, cutoff rule, model hash, scenario data, priors or weights, seed/stream scheme, solver settings, decimal and rounding conventions, calendars, environment and result locations. Absence of a usable Git revision is recorded as unavailable with content hashes, rather than inventing a commit.

Chapter 10

Acceptance and implementation sequence

10.1 Three different questions require different evidence

Engineering correctness asks whether the implementation follows the specified types, transitions and arithmetic. Accounting validity asks whether the selected rules faithfully implement the applicable standards and disclosed policies. Scientific validity asks whether the economic assumptions and uncertainty representation support the intended forecasts or comparisons. A balanced journal answers only part of the first question.

The implementation must therefore maintain separate validation results. Compilation and deterministic examples cannot establish forecasting quality. Agreement with a public report cannot identify an undisclosed contract term. A lower extraction error on a document benchmark cannot establish that the engine preserved every materially different scenario response.

10.2 Core conformance cases

Test	Required case	Pass condition
T01	Exact debit/credit batches, including reversals and period closing	Every committed book conserves signed balances; imbalance rejected atomically.
T02	Money, quantities, currencies, instants and interval flows	Invalid additions and context combinations rejected; explicit conversions traced.
T03	Original filing, later correction and as-known cutoff	Earlier query reproduces the original evidence view; later view shows revision.
T04	Annual and interim cumulative flows with revised comparatives	Quarter derivation only across compatible contexts and vintages.
T05	Fixed/floating borrowing with identical historical postings	Both explanations survive; next-year expense is 5 and 7 under the stated shock.

Test	Required case	Pass condition
T06	Continuous fixed share $\alpha \in [0, 1]$	Exact model returns $I = 7 - 2\alpha$ and extrema 5, 7 without assigning probabilities.
T07	Duplicated encoding of one history	Canonical duplicates preserve results and declared weights; no extra probability mass from an additional encoding.
T08	Linked interpretation across debt classification and payment treatment	Impossible combinations rejected through shared constraints.
T09	Rounded totals, duplicate facts and contradictory notes	Correct interval compatibility; no automatic zero or balancing plug.
T10	Infeasible or unsupported constraint problem	Conflicting sources exposed; timeout/unsupported case reported as unknown.
T11	Zero-coupon bond and ordinary equity examples	Carrying amounts, payments, share quantities and report mappings match derivations.
T12	Foreign-currency asset, translation and cash movement	No mixed-unit balance test; translation effects separated from cash flows.
T13	Acquisition and intragroup balance elimination	Perimeter and adjustment book reproduce group totals without inventing capex.
T14	Conflicting recognition rules and unknown guards	Explicit conflict or symbolic partition; file order has no effect.
T15	Insufficient cash with constrained funding	Funding failure/default retained as an outcome; no hidden borrowing.
T16	Expected-loss measurement before future default is realised	Measurement uses only information allowed at that date.
T17	Repeated/reordered independent events and interrupted execution	Idempotent replay; branch isolation; identical committed outputs.
T18	Conditional distributions and optional weighted mixture	Branch distributions retained; mixture agrees with declared weights and total probability; unweighted family never labelled probabilistic.

For synthetic cases, expected histories, excluded histories and scenario consequences are known

by construction. Tests should include counterexamples that balance but have the wrong economic effect, such as treating a non-cash acquisition movement as operating cash. Such tests discriminate between arithmetic validity and the intended semantics.

Differential tests against an existing ledger check only the shared supported subset. ACTUS schedule comparisons check contractual events, and QuantLib comparisons check selected valuation calculations under matched conventions. Disagreements must be classified as different conventions, unsupported cases or implementation errors before one tool is treated as an oracle.

10.3 Extraction and reconstruction evaluation

A company corpus must include different standards, industries, languages, filing formats, extensions, interim calendars and restatements. Company and time splits are required so repeated report templates or revised versions do not leak into evaluation. Public benchmark tables are useful development material, while acceptance uses complete disclosure packages that include notes and their relationships.

Reviewers annotate source facts, contexts, mappings, required relationships, known ambiguities and unsupported assertions. Relevant measures include correct source-linked values and contexts, unresolved extraction volume, coverage of note obligations, reconciliation failures and invented facts. Report each separately. A single overall score can hide a serious error in a small but consequential financing note.

Historical reconstruction should test whether the known synthetic history remains feasible, whether known incompatible histories are rejected, and whether economically distinct alternatives are preserved. On real companies, public data generally cannot establish the true journal. Review instead assesses evidential compatibility, unsupported commitments and conditional economic consequences. Later disclosures may provide discriminating evidence for previously ambiguous terms.

No universal numerical threshold for semantic coverage or extraction accuracy is fixed in this document. Before a release evaluation, an accounting-reviewed test corpus and task-specific acceptance thresholds must be recorded in an evidence contract. Hard conformance failures, fabricated citations, silent discarding of valid alternatives and undocumented accounting adjustments veto acceptance regardless of an aggregate score.

10.4 Evaluate the DSL against the strongest practical baseline

The comparison uses the same IR, accounting examples and tasks. One group of models is authored through the typed Python API; another through the DSL. Both receive equivalent documentation and examples. Tasks include encoding a movement schedule, adding an alternative interpretation, changing an effective date and tracing a simulated amount. Measure correctness, material omissions, time to a reviewed result, ease of explaining a change and maintenance effort.

This comparison tests whether the notation and compiler help users; it is not a comparison between careful DSL code and unstructured spreadsheets. If the DSL offers little advantage, retain the shared IR and library and narrow the textual language. Small pilots nominate improvements. A claim of superiority requires a predeclared comparison and uncertainty evidence, not a few favourable demonstrations.

10.5 Forecasting evaluation comes later

Forecast evaluation uses historical information cutoffs, fixed data vintages, declared economic scenarios and proper separation of training and testing periods. Baselines should include a naive persistence or ratio model, a tuned conventional financial model, the structural model with a single selected interpretation, and the same model preserving alternatives. This identifies whether any benefit comes from better economics, more information or the representation of ambiguity.

Construct the inexpensive comparators before choosing a complex forecasting method. A no-growth operating forecast with scheduled contractual financing tests whether the model adds value beyond existing obligations. A constant margin and turnover model tests whether detailed operations improve on familiar financial ratios. A trailing-average growth model tests whether fitted dynamics improve on recent experience. A fixed-exposure contractual stress model tests whether estimated management responses help beyond repricing the existing book. Each comparator must generate coherent accounts through the same reporting rules and use the same historical information. In synthetic cases, the known hidden state also supplies an oracle conditional on the stipulated scenario.

Evaluate these comparisons separately under stable conditions, large rate resets, refinancing pressure, credit deterioration and changes in reporting perimeter. A pooled average can conceal failure in the situation the product is intended to explain. Supported underperformance against a cheap comparator in a declared critical situation vetoes promotion of the complex forecasting method and must lead the result discussion. These checks are falsification tests, not training targets or evidence of superiority when merely passed.

Evaluate point forecasts where they are a stated target, distributional calibration when probabilities are justified, and decision-relevant robustness when only feasible sets are available. A wide feasible set is not automatically a good prediction, and an unweighted set cannot be assessed as if it were a 95% predictive interval. In multi-scenario stress tests, scenario severity and probability remain separate inputs.

Any serious numerical comparison must follow the repository's research plan, assumption audit, baseline, veto, uncertainty and result-record policies. The present document runs only deterministic checks of its own examples. It neither selects a scientific forecasting default nor reports comparative performance.

10.6 Implementation milestones and dependency order

Stage	Concrete implementation deliverable	Exit evidence
P0	Evidence registry, contextual types, versioned IR and exact posting engine; Python constructors for the running cases	T01–T04, T09 and T17; source inspection and replay.
P1	Finite choices, shared symbols, linear solver adapter, historical bridges, conditional simulation and report explanations	T05–T10, T14 and T18; synthetic family preserved.
P2	Textual Core 0 frontend with identical IR; initial corporate package; document/XBRL import adapters	Parser/API equivalence; T11–T13; reviewed three-date public-company case.

Stage	Concrete implementation deliverable	Exit evidence
P3	Contract and valuation adapters, dated standard packages, richer financing and tax/funding policies	Contract/valuation differential cases; accounting review; T15 and transition cases.
P4	Bank package, allowance models, regulatory reconciliation and management response alternatives	T16; banking-specific reviewed histories and stresses.
P5	Insurance cohort package, reinsurance and actuarial measurement adapters	Reviewed cohort roll-forwards, information-date and service/finance tests.
P6	Optional probabilistic inference, model-weight sensitivity and broader scenario research	Explicit targets, sampler/numerical validity, prior sensitivity and downstream evaluation.

These stages specify dependency order, not dates or staffing estimates. Banking and insurance representation requirements influence P0 types, although their measurement engines arrive later. Probability weighting is optional; P1 already produces useful conditional cases and bounds. A failure of a chosen extractor or inference algorithm triggers a local replacement or repair, rather than abandoning the product's representation requirements.

10.7 Decisions and replaceable assumptions

The requirements fixed by this design are preservation of evidence and alternatives, explicit time and basis, balanced and admissible posting, economic state beyond journals, and distinct set/probability semantics. Replaceable choices include the storage engines, parser generator, solver, numerical libraries, aggregation resolution and user interface. Every scientifically material aggregation, behavioural parameter, prior or truncation rule must record provenance, purpose, failure mode and a diagnostic before it becomes a default.

The immediate next implementation task is P0 with the synthetic financing cases and their evidence records. This provides a small testable foundation for the entire design. Broader corporate coverage should grow through reviewed examples, not through a claim that one generic rule package represents all financial statements.

Appendix A

Core grammar, records and execution details

A.1 Core 0 grammar

The grammar below defines the declaration forms used in this document. Identifiers are case-sensitive Unicode names resolved to namespace-qualified IDs. Keywords are reserved. Strings are UTF-8 JSON-style quoted strings. Comments start with `//` and end at the line boundary. Dates have the lexical form `@YYYY-MM-DD`; currency identifiers resolve through the package registry. Decimal literals have no implicit currency or rate type. Array and record expressions are immutable values. Functions must be registered pure functions with typed signatures; calling an unknown function is an error.

```
program      = "module" ident "version" string ";" { declaration } ;
declaration  = use | source | entity | book | account | observation
             | symbol | choice | binding | constraint | alternative
             | contract | event | rule | scenario | report ;
use          = "use" ident "version" string ";" ;
source       = "source" ident "=" expression ";" ;
entity       = "entity" ident "functional" currency ";" ;
book         = "book" ident "entity" ident "basis" ident
               "version" string ";" ;
account      = "account" ident ":" account_kind "book" ident ";" ;
account_kind = "Asset" | "Liability" | "Equity" | "Income" | "Expense" ;
observation  = "observation" ident ":" type time_spec
               "=" expression "from" ident ";" ;
time_spec    = "instant" date | "period" date "to" date ;
symbol       = "symbol" ident ":" type "in" domain
               "because" string ";" ;
domain       = "[" expression "," expression "]"
             | "{" expression { "," expression } "}" ;
choice       = "choice" ident "in" "{" ident { "," ident } "}"
               "because" string ";" ;
binding      = "let" ident [ ":" type ] "=" expression ";" ;
constraint   = "constraint" ident ":" expression ";" ;
alternative  = "alternative" ident "{" { assumption } "}" ;
assumption   = "assume" expression ";" ;
contract     = "contract" ident ":" ident "book" ident
               "{" { field } "}" ;
field        = ident "=" expression ";" ;
event        = "event" ident "(" [ parameters ] ")" "book" ident ";" ;
```

```
parameters   = ident ":" type { "," ident ":" type } ;
rule         = "rule" ident "for" ident "as" ident
               [ "overrides" ident { "," ident } ]
               "when" expression "{" { rule_statement } "}" ;
rule_statement = binding | "require" expression ";" | post ;
post         = "post" "{" { posting } "}" ;
posting      = ("debit" | "credit") ident expression ";" ;
scenario     = "scenario" ident "{" { "set" ident "=" expression ";" } "}" ;
report       = "report" ident "for" ident "{" { field } "}" ;
type         = ident [ "<" type { "," type } ">" ] ;
expression   = disjunction ;
disjunction  = conjunction { "or" conjunction } ;
conjunction  = comparison { "and" comparison } ;
comparison   = sum [ ("==" | "!=" | "<" | "<=" | ">" | ">=") sum ] ;
sum          = product { ("+" | "-") product } ;
product      = unary { ("*" | "/") unary } ;
unary        = [ "-" | "not" ] atom ;
atom         = number [ currency | "per" "year" ] | string | date
             | "true" | "false" | ident { "." ident }
             | ident "(" [ expression { "," expression } ] ")"
             | "(" expression ")"
             | "[" [ expression { "," expression } ] "]"
             | "{" { field } "}" ;
```

The `period` form's second date is exclusive. Symbol-domain intervals are closed in Core 0; open domains can be expressed by additional constraints. Money and rate suffixes have lexical priority over unrelated identifiers. The parser rejects ambiguous tokenisation, duplicate names, unresolved currency identifiers and chained comparisons. An identifier introduced by a choice is an enumeration value within that choice's namespace.

Core 0 has no unbounded loops, recursion, dynamic code evaluation or arbitrary host calls. Contract schedules and finite input collections are supplied by typed packages. The `report` fields bind output names to typed expressions provided by a registered reporting template; they do not redefine the recognition of a transaction. Time-dependent scenario drivers are typed curves or processes; the illustrative `constant` function supplies a constant curve over the requested run horizon.

A.2 Required core record definitions

The following language-neutral definitions specify serialised meanings. An implementation may use classes, algebraic data types or relational tables, provided round-trip import/export preserves these fields.

```
EvidenceView {
  id, source_document_ids[], extraction_version_ids[],
  cutoff: Timestamp, availability_policy_id,
  revision_policy: Original | LatestAvailable | ExplicitSelection,
  selected_assertion_version_ids[], review_decision_ids[]
}

Context {
  entity_id, perimeter_version_id?, concept_id, basis_id,
  time: Instant(Date) | Interval(start: Date, end_exclusive: Date),
  unit_id, currency_id?, dimensions: Map<DimensionId, MemberId>,
  taxonomy_version_id?, measurement_kind
}
```

```
ObservedValue<T> =
  Exact(T) |
  Rounded(displayed: T, admissible_domain: Domain<T>, rule_id) |
  Missing(reason: MissingReason)

Observation<T> {
  id, context_id, value: ObservedValue<T>, source_span_ids[],
  literal, extraction_run_id, proposed_reading_id,
  review_status, supersedes_id?
}
```

```
Assumption {
  id, expression_id, scope_id, rationale,
  provenance: Analyst | EmpiricalEstimate | ImportedModel | ScenarioDesign,
  source_refs[], valid_times, information_cutoff,
  role: Baseline | Hypothesis | Convenience | ReviewedDefault
}

HistoryFamily {
  id, model_scope_id, evidence_view_id, opening_state_spec,
  shared_symbols[], finite_choices[], constraints[], alternatives[],
  feasibility: Feasible | Infeasible | Unknown,
  coverage: ExactWithinScope | PartialSearch | Approximate,
  solver_result_refs[], witnesses[]
}
```

```
Posting {
  account_id, side: Debit | Credit,
  functional_amount: NonnegativeExactMoney,
  original_quantity?, original_currency?, valuation_ref?,
  movement_class, counterparty_id?, lot_or_cohort_id?
}

JournalBatch {
  id, event_id, book_id, posting_date, accrual_period?,
  postings[], rule_version_id, measurement_refs[],
  assumption_refs[], state_precondition_hash, explanation_ref
}
```

```
ConditionalOutput {
  context_id, history_family_id, condition_refs[], scenario_id,
  result: Exact | Symbolic | Samples | CertifiedBounds
          | ExploredRange | Unresolved,
  probability_model_id?, numerical_status, completeness,
  explanation_refs[]
}
```

Question marks denote explicitly optional fields. An absent field is allowed only when the type's conditions permit it; for example, monetary output always has a currency. A posted functional amount is exact and nonnegative, with side carrying the sign. Zero postings may be retained for traceability, but cannot serve as invented evidence of an economic event. The underlying measurement can have finer precision or a continuous symbolic domain before the declared posting-rounding operation.

A.3 Typing and partial-information rules

For expressions x and y in the same currency C, the money addition rule is

$$\frac{x : \text{Money}\langle C\rangle \quad y : \text{Money}\langle C\rangle}{x + y : \text{Money}\langle C\rangle}.$$

Context compatibility is a separate premise for observations. There is no analogous implicit rule for different currencies. Explicit conversion uses a dated measurement and records its rate and purpose. A rate-times-principal calculation is typed through a declared accrual convention; the example's annual simple rate and year fraction produce money in the principal's currency.

Known expressions evaluate normally. Shared symbols evaluate to symbolic expressions subject to a joint constraint store. Missing source values cannot enter arithmetic until the model supplies a symbol or a conditional assumption. Boolean guards over symbols produce formulas, not the value false. This avoids the common error in which "unknown whether impaired" silently means "not impaired." Division requires a proven nonzero denominator or a partitioned domain that excludes zero; otherwise the expression remains unsupported for concrete execution.

Logical queries first require a nonempty feasible family. An empty family returns `Infeasible`; it must not produce vacuous answers to substantive questions. For a nonempty family, constraint evaluation has three results. `entailed` means every state in the declared feasible family satisfies the proposition. `refuted` means none does. `undetermined` means both remain feasible or the solver has not decided; the response distinguishes those causes. Probabilistic truth is a separate query requiring a measure. A language-level Boolean value must never be inferred from an arbitrary probability threshold.

A.4 Operational rule selection

Let $V(e, h)$ be the applicable rules for event e in concrete history h, after package/date selection and guard evaluation. Let $r \prec r'$ mean that r' validly overrides r through the declared acyclic graph. Define

$$\text{Max}(V) = \{r \in V : \nexists r' \in V \text{ with } r \prec r'\}.$$

For one required recognition operation, a singleton selects that rule; the empty set yields `UnsupportedTreatment`; a larger set yields `RuleConflict`. Multiple distinct operations, such as

measurement and recognition, are evaluated through separate declared interfaces. A rule that intentionally produces no postings must belong to a supported non-recognition event operation, whose economic state effect is still recorded.

Given a selected rule, the evaluator computes all local bindings and preconditions, emits proposed postings, verifies Equation (5.2), validates account/context restrictions and commits the event transaction. Failure returns the original state plus a diagnostic. This defines the atomicity requirement independently of the storage technology.

A.5 Example run request and response

The following request executes two named conditions under one scenario. It does not request an aggregate probability distribution.

```
{
  "model": "BorrowingExample@1",
  "evidence_view": "synthetic-2025-exact",
  "starting_state": "after-2025-close",
  "horizon": {"start": "2026-01-01", "end_exclusive": "2027-01-01"},
  "conditions": ["allFixed", "allFloating"],
  "scenarios": ["rateShock"],
  "mode": "conditional",
  "weight_model": null,
  "reports": ["balance_sheet", "income", "cash_flow", "equity", "journals"],
  "package_lock": "demonstration-packages-v1"
}
```

The response contains two separately labelled results with interest expense 5 and 7, cash 90 and 88, debt 100 in both, and equity -10 and -12. A request for the parent symbolic family returns the formula $7 - 2\alpha$ and, under exact linear bounds, $[5, 7]$ before monetary quantisation. The API rejects an unconditional mean request without a weight model. A user can instead request an arbitrary clearly labelled representative case, which does not stand in for the feasible family.

A.6 Requirement traceability

Requirement	Primary implementation area	Conformance cases
R1	Evidence registry and extraction adapters	T03, T09
R2	Context/version model and cutoff queries	T03, T04
R3	Types, mappings and package selection	T02, T12, T13
R4	Symbols, choices and constraint graph	T05–T10, T14
R5	Posting, closing and consolidation	T01, T11–T13
R6	Contract and behavioural interfaces	T05, T11, T15, T16
R7	Conditional output and probability types	T06, T07, T18
R8	Explanation graph and report dependencies	T09, T11–T13
R9	Capabilities and versioned modules	T02, T14–T16
R10	Run records, scheduler and replay	T03, T17, T18

Appendix B

Research basis and outstanding review

B.1 Source collection and reading scope

The companion library contains 31 verified paper PDFs: 24 retrieved from external sources and seven preserved copies from the original collection. Filenames use the title, first author's surname and year. Publication years are used where an archived working version underlies a later publication; the catalogue identifies the actual version read. The original collection has not been renamed or modified. Standards pages, contract specifications, software source snapshots and metadata are stored separately from papers.

The survey considered accounting algebra and DSLs, corporate and bank simulation, incomplete and temporal databases, provenance, inverse problems, partial identification, model averaging, causal scenarios, contract languages, probabilistic programming, document extraction and financial reasoning. OpenAlex and Crossref supplied dated metadata, while primary publisher, institutional and author copies supplied technical support. Public-index results were checked against the downloaded title pages; HTTP success alone was not accepted as a valid paper download.

Technical reading was targeted to the mechanisms used in this design. The full PDF is archived even where only specified sections support the argument. No claim is made that every theorem, appendix or empirical result in every archived paper was independently replicated. Representative support anchors are listed below; the source-support ledger contains the per-source scope and qualifications.

Source	Inspected technical support	Design use
Nehmer and Robinson	Sections 2–5: account states, transactions and aggregation	Conservation plus admissibility.
Doshi	Chapters 4, 7–9 of the 2024 thesis	Dates, logs, corrections and DSL scope.
Shahnazarian	Sections 2.3–2.8; relevant Chapter 3 equations; data and evaluation sections; Appendix B	Accounting/behaviour separation; limits of data transfer.
Pierson	Sections 3–4 and reporting structures; supplementary Vensim equations	Accrual/payment bridges and reporting accumulators.

Source	Inspected technical support	Design use
Libkin	Sections 2–5: incomplete states, conditions and certain answers	Shared unknowns and joint alternatives.
Green et al.	Definitions 3.1–3.2; Section 4	Composable relational provenance; limits outside positive operations.
Stuart	Section 2.2, observation equation and prior/regularisation discussion	Inverse-problem formulation, not imported convergence guarantees.
Giacomini and Kitagawa	Sections 2.1–2.6 and Theorem 1 in CWP12/20	Prior sensitivity and classes of posterior beliefs.
Hoeting et al.	Equations (1)–(3); prior and predictive-evaluation discussion	Optional model-weighted predictions.
Merigoux et al.	Sections 3–4, default calculus and translation figures	Scopes, exceptions, empty/conflict outcomes.
Peyton Jones et al.	Sections 3–5, combinators and valuation semantics	Contract composition separate from valuation assumptions.
FinQA / TAT-QA	Dataset operations, annotation, model and evaluation sections; FinQA evaluation source	Evidence-linked local arithmetic; benchmark boundaries.
DocFinQA / FinBen	Context/retrieval and financial-task evaluations	Long-document difficulty and task coverage.
PubTables / GTE	Canonicalisation, dataset construction, methods and evaluation	Table structure and financial-domain extraction.
Cosentino et al.	Sections 3.1–3.2, 4.6 and 5	Persistent RAG provenance; scope of auditability evidence.

B.2 Version and access qualifications

Several discrepancies mattered. Doshi's thesis is dated 2024 despite the original filename. The archived Mernik text is the 2003 CWI report underlying the 2005 article. Giacomini and Kitagawa's copy is the April 2020 working paper underlying the 2021 article. The Docling v5 copy lists Auer as first author. The probabilistic-programming tutorial is an arXiv version revised in 2021, first submitted in 2018; a merged index DOI was not used. The Libkin author PDF has a footer DOI that resolves to another paper; title/author metadata from the indexes resolves the cited work to DOI `10.1145/2594538.2594561`. These discrepancies remain documented rather than being hidden by a cleaned bibliography.

McCarthy (1982), Imieliński and Lipski (1984), Tamer (2010), and Caiani and co-authors (2016) are retained as important blocked candidates. Access failures and rejected wrong-content downloads are recorded. The design does not use their uninspected technical claims. More specialised work on accounting measurement, process mining, multi-company equilibrium, actuarial inference and learned forecasting is recorded as expansion work for the corresponding implementation stage. The survey supports the core architecture without pretending that those later modules have a complete literature or rule base.

OpenAlex retraction flags and available Crossref correction/version relations were inspected

alongside local versions. No reviewed source was excluded for a discovered retraction. Missing correction metadata is marked unavailable; absence of a discovered notice does not certify that no notice exists. Citation counts are dated index metadata, and venue rankings were not available in this campaign. Neither is used as evidence of correctness.

B.3 Companion records and status

The six research ledgers are `source-support.json`, `citation-venue.json`, `backward-snowball.json`, `forward-snowball.json`, `claim-support.json`, and `omissions.json`. They record technical anchors, metadata sources, considered leads, omitted or blocked candidates and the distinction between source claims and this document's derivations. Citation leads with checked seed references are distinguished from cross-survey additions and topic expansion leads. Forward searches retain the most-cited and most-recent results returned by the recorded bounded queries; they do not establish exhaustive citation coverage. The separate `tools.json` records software evidence. Raw retrievals and text extracts permit a later reviewer to revisit the decisions.

This document specifies the product; it does not report a completed engine. The companion review records cover deterministic example arithmetic, bibliography/build checks, rendered-page inspection and author review. Human comprehension review, independent accounting review and implementation conformance remain separate future evidence. They are not replaced by a clean LaTeX build or by the quantity of literature collected.

Bibliography

ACTUS Financial Research Foundation. ACTUS technical specifications: contract states, events, payoffs and risk-factor interface, 2026. URL https://github.com/actusfrf/actus-techspecs. Official source, accessed 2 October 2026.

Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, et al. Docling Technical Report. Technical report, arXiv:2408.09869v5, 2024. URL https://arxiv.org/pdf/2408.09869.

Egor Babcinețchi, Daniel Pogorevici, Iulia Țăruș, and Rafaela Cerlat. Domain Specific Language for Accounting. Technical University of Moldova student research conference, pp. 342–345, 2022.

Markus Behn, Claudio Daminato, and Carmelo Salleo. A dynamic model of bank behaviour under multiple regulatory constraints. European Central Bank Working Paper 2233, 2019. URL https://doi.org/10.2139/ssrn.3327174. ECB Working Paper 2233; local DOI 10.2866/943267. OpenAlex SSRN record used only for citation metadata.

Eli Bingham, Jonathan P. Chen, Martin Jankowiak, et al. Pyro: Deep Universal Probabilistic Programming. Journal of Machine Learning Research, 20(28):1–6, 2019. URL https://jmlr.org/papers/volume20/18-403/18-403.pdf.

Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, et al. Stan : A Probabilistic Programming Language. Journal of Statistical Software, 76(1), 2017. URL https://www.jstatsoft.org/index.php/jss/article/view/v076i01/v76i01.pdf.

Zhiyu Chen, Wenhu Chen, Charese Smiley, et al. FinQA: A Dataset of Numerical Reasoning over Financial Data. Proceedings of EMNLP, 2021. URL https://aclanthology.org/2021.emnlp-main.300.pdf.

Cristian Cosentino, Simone Squillace, and Fabrizio Marozzo. An Auditable LLM-RAG Architecture for Financial Document Intelligence and Decision Support. Future Internet, 18, article 284, 2026. URL https://mdpi-res.com/d_attachment/futureinternet/futureinternet-18-00284/article_deploy/futureinternet-18-00284.pdf.

Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. Proceedings of TACAS, 2008. URL https://link.springer.com/content/pdf/10.1007%2F978-3-540-78800-3_24.pdf.

Janhavi Doshi. A Domain-Specific Language for Accounting. Master thesis, Rochester Institute of Technology, 2024. Title page 2024; original user filename incorrectly suggests 2026.

Raffaella Giacomini and Toru Kitagawa. Robust Bayesian Inference for Set-Identified Models. Econometrica, 89; archived CWP12/20 working version, 2021. URL https://ifs.org.uk/sites/default/files/output_url_files/CW1220-Robust-Bayesian-inference-for-set-identified-models.pdf. CWP12/20, 8 April 2020 working paper underlying 2021 Econometrica article; technical claims anchored to this local version.

Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. Proceedings of PODS, 2007. URL https://repository.upenn.edu/bitstreams/b598c0a7-0d24-4162-8279-5f51a17d29c2/download.

Grzegorz Halaj. Dynamic balance sheet model with liquidity risk. European Central Bank Working Paper 1896, 2016. URL https://doi.org/10.1142/s0219024916500527. ECB Working Paper 1896, April 2016. Index journal DOI is not asserted to be the version read.

Jennifer A. Hoeting, David Madigan, Adrian E. Raftery, and Chris T. Volinsky. Bayesian Model Averaging - A Tutorial. Statistical Science, 14(4):382–417, 1999. URL https://www.stat.colostate.edu/~jah/papers/statsci.pdf.

IFRS Foundation. IAS 32 Financial Instruments: Presentation: issued-standard overview, 2026a. URL https://www.ifrs.org/issued-standards/list-of-standards/ias-32-financial-instruments-presentation/. Official source, accessed 2 October 2026.

IFRS Foundation. IFRS 17 Insurance Contracts: issued-standard overview, 2026b. URL https://www.ifrs.org/issued-standards/list-of-standards/ifrs-17-insurance-contracts/. Official source, accessed 2 October 2026.

IFRS Foundation. IFRS 18 Presentation and Disclosure in Financial Statements: issued-standard overview, 2026c. URL https://www.ifrs.org/issued-standards/list-of-standards/ifrs-18-presentation-and-disclosure-in-financial-statements/. Official source, accessed 2 October 2026.

IFRS Foundation. IFRS 9 Financial Instruments: issued-standard overview, 2026d. URL https://www.ifrs.org/issued-standards/list-of-standards/ifrs-9-financial-instruments/. Official source, accessed 2 October 2026.

Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, Ming Zhou, and Zhoujun Li. TableBank - A Benchmark Dataset for Table Detection and Recognition. Proceedings of LREC; arXiv:1903.01949, 2020. Metadata search returned original 2019 preprint; local version title is TableBank: A Benchmark Dataset for Table Detection and Recognition.

Leonid Libkin. Incomplete Data - What Went Wrong, and How to Fix It. Proceedings of PODS, 2014. URL https://homepages.inf.ed.ac.uk/libkin/papers/pods14.pdf. Author PDF footer prints 10.1145/2594538.2594551, which indexes another paper. Title/author and Crossref bibliography resolve this paper to 10.1145/2594538.2594561. The citation uses the corrected metadata and author-hosted text; discrepancy retained.

Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: a programming language for the law. Proceedings of the ACM on Programming Languages, 5(ICFP), 2021. URL https://arxiv.org/pdf/2103.03198.

Marjan Mernik, Jan Heering, and Anthony M. Sloane. When and how to develop domain-specific languages. ACM Computing Surveys; archived CWI SEN-E0309 (2003), 2005. URL https://ir.cwi.nl/pub/4109/04109D.pdf. Local full text is CWI report SEN-E0309, December 2003; publication 2005. Claims anchored to the archived working paper.

Robert A. Nehmer and Derek Robinson. An algebraic model for the representation of accounting systems. Annals of Operations Research, 71:179–198, 1997. URL https://doi.org/10.1023/a:1018915430594.

Judea Pearl. Causal inference in statistics: An overview. Statistics Surveys, 3:96–146, 2009. URL https://ftp.cs.ucla.edu/pub/stat_ser/r350.pdf.

Simon Peyton Jones, Jean-Marc Eber, and Julian Seward. Composing contracts: an adventure in financial engineering. Proceedings of ICFP; author manuscript dated 23 August 2000, 2000. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/contracts-icfp.pdf.

Kawika Pierson. Operationalizing Accounting Reporting in System Dynamics Models. Systems, 8(1), article 9, 2020. URL https://doi.org/10.3390/systems8010009.

Varshini Reddy, Rik Koncel-Kedziorski, Viet Dac Lai, Michael Krumdick, Charles Lovering, and Chris Tanner. DocFinQA: A Long-Context Financial Reasoning Dataset. Proceedings of ACL, short papers, 2024. URL https://aclanthology.org/2024.acl-short.42.pdf.

Hovick Shahnazarian. A Dynamic Microeconometric Simulation Model for Incorporated Businesses. Sveriges Riksbank Occasional Paper 11, 2004.

Brandon Smock, Rohith Pesala, and Robin Abraham. PubTables-1M: Towards comprehensive table extraction from unstructured documents. Proceedings of CVPR; author manuscript, 2022. URL https://arxiv.org/pdf/2110.00061.

Richard Snodgrass and Ilsoo Ahn. Temporal Databases. Computer, 19(9):35–42, 1986. URL https://www2.cs.arizona.edu/~rts/pubs/Computer.pdf. Corrected after title-page inspection: author link Computer.pdf is the 1986 paper, not Jensen and Snodgrass 1999.

Andrew M. Stuart. Inverse problems: A Bayesian perspective. Acta Numerica, 19:451–559, 2010. URL https://authors.library.caltech.edu/records/fk9j9-5tq15/files/stuart15c.pdf.

Jan-Willem van de Meent, Brooks Paige, Hongseok Yang, and Frank Wood. An Introduction to Probabilistic Programming. arXiv:1809.10756; version 2 (2021), 2018. URL https://arxiv.org/pdf/1809.10756. arXiv v2 dated 19 October 2021; first submission 2018; OpenAlex merged DOI belongs to another venue and is not used.

Arie van Deursen, Paul Klint, and Joost Visser. Domain-Specific Languages. ACM SIGPLAN Notices, 35(6):26–36, 2000. URL https://homepages.cwi.nl/~paulk/publications/Sigplan00.pdf.

XBRL International. Open Information Model 1.0: Recommendation, 13 October 2021, 2021. URL https://www.xbrl.org/Specification/oim/REC-2021-10-13/oim-REC-2021-10-13.html. Official source, accessed 2 October 2026.

Qianqian Xie, Weiguang Han, Zhengyu Chen, et al. FinBen: A Holistic Financial Benchmark for Large Language Models. NeurIPS Datasets and Benchmarks; arXiv:2402.12659, 2024. URL https://arxiv.org/pdf/2402.12659.

Xinyi Zheng, Douglas Burdick, Lucian Popa, Xu Zhong, and Nancy Xin Ru Wang. Global Table Extractor (GTE): A Framework for Joint Table Identification and Cell Structure Recognition Using Visual Context. Proceedings of WACV, 2021. URL https://openaccess.thecvf.com/content/WACV2021/papers/Zheng_Global_Table_Extractor_GTE_A_Framework_for_Joint_Table_Identification_WACV_2021_paper.pdf.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance. Proceedings of ACL-IJCNLP, 2021. URL https://aclanthology.org/2021.acl-long.254.pdf.